



UNIVERSITÀ DEGLI STUDI DELL'INSUBRIA

Dipartimento di Scienze Teoriche e Applicate

Laurea Triennale in Informatica

Corso: Interfacce Uomo-Macchina – a.a. 2025/26

Docente: Prof.ssa Silvia Corchs

Documentazione Tecnica

Pipeline EEG-IGT

Behavioral and electroencephalographic dataset
simultaneously acquired during the Iowa Gambling Task

Dataset: Chávez-Sánchez M., Torres-Ramos S.,
Román-Godínez I., Salido-Ruiz R.A.

Autori: *Babini Ariele - 757608*
Bottaro Federico - 758017
Como Nicholas - 757386

Rivista: Scientific Data, 13:359, 2026

DOI: 10.1038/s41597-026-06662-0

Dataset: Mendeley Data – 2pw2m39yct, versione 2

Progetto d'esame – Documentazione tecnica completa (Parti I, II e Orchestratore)
Maggio 2026

Contents

1	Introduzione e Obiettivi	4
1.1	Contesto del progetto	4
1.2	Obiettivo della classificazione	4
1.3	Struttura della documentazione	5
2	Dataset	5
2.1	Descrizione generale	5
2.2	Descrizione dei gruppi sperimentali	6
2.3	Protocollo sperimentale	6
2.4	Iowa Gambling Task – Struttura dei payoff	7
2.5	Struttura dei file del dataset	7
2.6	Canali EEG e posizionamento sistema 10-20	8
3	Architettura Generale del Sistema	9
3.1	Panoramica dell'architettura a tre livelli	9
3.2	Costanti globali e parametri configurabili	10
3.3	Dipendenze software	11
3.4	Struttura dei moduli e funzioni esportate	12
4	Pipeline di Preprocessing EEG (eeg_preprocessing.py)	13
4.1	Struttura del modulo e funzione principale	13
4.2	Step 1 – Caricamento dei file sorgente	14
4.3	Step 2 – Creazione oggetto MNE RawArray	14
4.4	Step 3 – Re-referencing: Common Average Reference	15
4.5	Step 4 – Filtraggio Bandpass e Notch	16
4.6	Step 5 – Rimozione Artefatti con ICA	18
4.7	Step 6 – Normalizzazione Z-Score per Canale	20
5	Sincronizzazione EEG-IGT ed Estrazione Epoche	21
5.1	Meccanismo di sincronizzazione – <code>sync_eeg_igt()</code>	21
5.2	Validazione dei trial	21
5.3	Generazione delle label binarie	21
5.4	Estrazione delle epoche – <code>extract_epochs()</code>	22
6	Output del Preprocessing e Formato per ML	23
6.1	File salvati per soggetto – <code>save_subject_output()</code>	23
6.2	File riepilogativo di pipeline – <code>pipeline_summary.csv</code>	23
6.3	Caricamento per classificazione	23
6.4	Funzione <code>build_dataset_dataframe()</code>	24
7	Funzioni di Visualizzazione Diagnostica	24
7.1	<code>plot_raw_vs_clean()</code>	24
7.2	<code>plot_psd()</code>	24
7.3	<code>plot_epoch_mean()</code>	25
8	Pipeline di Feature Extraction PSD (eeg_ml_pipeline.py)	25
8.1	Panoramica e motivazione della scelta PSD	25
8.2	Algoritmo di Welch – <code>extract_psd_features()</code>	26
8.3	Costruzione della feature matrix – <code>build_feature_matrix()</code>	28
8.4	Struttura e nomi delle feature – <code>get_feature_names()</code>	29

9	Definizione dei Modelli di Machine Learning	29
9.1	Logistic Regression – baseline lineare	29
9.2	Support Vector Machine – kernel RBF	30
9.3	Random Forest – ensemble bagging	31
9.4	Pipeline sklearn e prevenzione del data leakage	32
10	Validazione: Leave-One-Subject-Out (LOSO)	32
10.1	Motivazione della strategia LOSO	32
10.2	Implementazione con LeaveOneGroupOut	33
10.3	Parallelizzazione dei fold – <code>run_cross_validation()</code>	33
10.4	GroupKFold come alternativa	34
11	Metriche di Valutazione	35
11.1	Metriche calcolate per fold	35
11.2	Aggregazione e tabella riepilogativa – <code>compute_summary_table()</code>	36
12	Visualizzazioni dei Risultati ML	36
12.1	Confusion Matrix aggregata – <code>plot_confusion_matrices()</code>	36
12.2	ROC Curve media con deviazione standard – <code>plot_roc_curves()</code>	37
12.3	Feature Importance – <code>plot_feature_importance()</code>	37
12.4	Distribuzione metriche per fold – <code>plot_per_fold_metrics()</code>	38
13	Generazione Dati Sintetici – <code>generate_synthetic_data()</code>	38
13.1	Motivazione e struttura	38
13.2	Struttura dei dati sintetici	38
14	Caricamento e Salvataggio Modelli – <code>load_preprocessed_data()</code> e <code>save_models()</code>	39
14.1	Caricamento – <code>load_preprocessed_data()</code>	39
14.2	Salvataggio modelli finali – <code>save_models()</code>	40
15	Orchestratore Unificato (<code>run_pipeline.py</code>)	40
15.1	Panoramica e motivazione architetturale	40
15.2	Configurazione del logging unificato	41
15.3	Funzione <code>_section(title)</code>	41
15.4	Step 0 – Generazione dataset sintetico	41
15.5	Step 1 – Preprocessing	42
15.6	Step 2 – Machine Learning	42
15.7	Funzione <code>main()</code> – Parsing argomenti e flusso principale	43
15.8	Struttura delle directory di output	44
16	Comandi di Esecuzione della Pipeline	45
16.1	Prerequisiti e posizionamento	45
16.2	Orchestratore unificato – uso raccomandato	45
16.3	Riepilogo argomenti dell’orchestratore	47
16.4	Uso diretto dei moduli (avanzato)	47
16.5	Riepilogo argomenti <code>eeg_ml_pipeline.py</code> (uso standalone)	48
16.6	File di output prodotti dalla pipeline ML	49
17	Risultati Ottenuti e Confronto con la Letteratura	49
17.1	Statistiche del dataset preprocessato	49
17.2	Variabilità inter-soggetto nella distribuzione delle scelte	49
17.3	Risultati LOSO – media $\pm$ std su 55 fold	50
17.4	Confronto con la letteratura EEG-IGT	51

18 Scelte Implementative e Motivazioni	51
18.1 PSD invece di analisi ERP per la classificazione	51
18.2 Classificazione binaria A/B vs C/D	51
18.3 Gestione dello sbilanciamento: <code>class_weight='balanced'</code>	52
18.4 Parallelizzazione con backend <code>loky</code>	52
18.5 Finestra pre-decisionale <code>[-2s, 0s]</code>	52
18.6 ICA: 3000 iterazioni e tolleranza 10^{-3}	52
18.7 Filtro notch con metodo FIR	52
18.8 Assenza di baseline correction	52
18.9 Libreria MNE-Python come framework EEG	52
18.10 Fallback <code>sklearn.utils.parallel / joblib</code>	53
19 Conclusioni e Sviluppi Futuri	53
19.1 Riepilogo	53
19.2 Sviluppi futuri	53

1 Introduzione e Obiettivi

1.1 Contesto del progetto

Il presente documento costituisce la documentazione tecnica unificata del progetto d'esame del corso di Interfacce Uomo-Macchina (a.a. 2025/26) e integra le due parti sviluppate separatamente (Parte I: preprocessing EEG; Parte II: feature extraction e machine learning) in un unico riferimento tecnico completo, aggiungendo la documentazione dell'orchestratore unificato `run_pipeline.py`.

Il progetto ha come obiettivo lo sviluppo di una pipeline software end-to-end per la **classificazione automatica delle decisioni umane nell'Iowa Gambling Task (IGT)** a partire dal segnale elettroencefalografico (EEG). Il problema è inquadrato come un task di classificazione binaria trial-by-trial su dati fisiologici cross-subject, con tutti i relativi vincoli metodologici (variabilità inter-soggetto, rischio di data leakage, necessità di cross-validazione rigorosa).

Il sistema è composto da tre moduli Python autonomi e componibili, ciascuno dei quali copre una fase specifica del flusso di analisi:

- `eeg_preprocessing.py` – Parte I: caricamento, pulizia e strutturazione del segnale EEG grezzo tramite libreria MNE-Python;
- `eeg_ml_pipeline.py` – Parte II: estrazione delle feature spettrali mediante Power Spectral Density (metodo di Welch) e classificazione con tre algoritmi di machine learning (Logistic Regression, SVM-RBF, Random Forest);
- `run_pipeline.py` – Orchestratore: coordina in sequenza tutte le fasi in un singolo comando da terminale con output terminale premium via libreria `rich`.

Il preprocessing costituisce la fase fondante dell'intera pipeline: la qualità del segnale elaborato determina direttamente le performance dei classificatori a valle. Un segnale non correttamente filtrato, non dereferenziato o contaminato da artefatti produce feature spurie che degradano qualsiasi modello di machine learning, indipendentemente dalla sua sofisticazione algoritmica.

1.2 Obiettivo della classificazione

Task di classificazione binaria trial-by-trial

Input: epoche EEG di shape `(n_epochs, 21, 513)` – segnale nella finestra temporale `[-2s, 0s]` rispetto al momento della decisione.

Output: etichetta binaria per ogni trial (una decisione = un vettore di 105 feature)

- **Classe 0 – Decisione svantaggiosa:** scelta del deck A o B (alto guadagno immediato, perdita netta a lungo termine)
- **Classe 1 – Decisione vantaggiosa:** scelta del deck C o D (guadagno moderato, bilancio positivo a lungo termine)

Feature: potenza spettrale PSD Welch in 5 bande $(\delta, \theta, \alpha, \beta, \gamma)$ su 21 canali $\Rightarrow$ vettore da 105 elementi per trial.

La distinzione tra scelte vantaggiose e svantaggiose riflette la dicotomia fondamentale dell'Iowa Gambling Task introdotta da Bechara et al. (1994): i deck A e B offrono premi immediati elevati ma producono perdite nette a lungo termine (comportamento impulsivo, associato a ridotta attivazione prefrontale ventromediale), mentre i deck C e D offrono premi moderati ma un bilancio cumulativo positivo (comportamento ottimale, associato ad apprendimento somatico). Un soggetto sano che sviluppa la strategia ottimale tende progressivamente a preferire C e D dopo circa 40–60 trial.

1.3 Struttura della documentazione

Il documento è organizzato nelle seguenti sezioni principali, che seguono il flusso logico dell’elaborazione del dato:

1. Sezione 2: descrizione dettagliata del dataset (paper, protocollo sperimentale, struttura file, canali EEG);
2. Sezione 3: architettura generale del sistema, costanti globali e dipendenze software;
3. Sezione 4: pipeline di preprocessing EEG – `eeg_preprocessing.py` (6 step);
4. Sezione 5: sincronizzazione EEG-IGT, validazione trial ed estrazione epoche;
5. Sezione 6: output del preprocessing, formato `.npy` e `pipeline_summary.csv`;
6. Sezione 7: funzioni di visualizzazione diagnostica del preprocessing;
7. Sezione 8: pipeline di feature extraction PSD – `eeg_ml_pipeline.py`;
8. Sezione 9: definizione e configurazione dei modelli di machine learning;
9. Sezione 10: strategia di validazione Leave-One-Subject-Out (LOSO) e GroupKFold;
10. Sezione 11: metriche di valutazione e aggregazione dei risultati;
11. Sezione 12: visualizzazioni dei risultati ML (CM, ROC, Feature Importance, Boxplot);
12. Sezione 13: generazione di dati sintetici per testing senza dataset reale;
13. Sezione 14: caricamento e salvataggio modelli finali;
14. Sezione 15: orchestratore unificato – `run_pipeline.py`;
15. Sezione 16: comandi di esecuzione della pipeline;
16. Sezione 17: risultati ottenuti e confronto con la letteratura;
17. Sezione 18: scelte implementative e motivazioni;
18. Sezione 19: conclusioni e sviluppi futuri.

2 Dataset

2.1 Descrizione generale

Il dataset è stato pubblicato da Chávez-Sánchez et al. (2026) su *Scientific Data* ed è pubblicamente disponibile su Mendeley Data (identificativo `2pw2m39yct`, versione 2). Contiene registrazioni simultanee di segnale elettroencefalografico e dati comportamentali raccolte su 59 partecipanti non clinici durante l’esecuzione dell’Iowa Gambling Task.

Il dataset è stato acquisito presso l’Universidad de Guadalajara (Messico) e rappresenta uno dei pochi dataset pubblici con registrazione EEG simultanea durante l’IGT. La sua pubblicazione su *Scientific Data* (rivista peer-reviewed del gruppo Nature) garantisce la validazione della qualità e della riproducibilità dei dati tramite il processo di data peer review.

Table 1: Caratteristiche principali del dataset di Chávez-Sánchez et al. (2026)

Parametro	Valore
Numero partecipanti	59 (30 maschi, 29 femmine)
Età media	21.44 anni ($\sigma = 1.30$)
Gruppo Engineering (ENG)	$n = 29$ (Ingegneria)
Gruppo Physical Culture and Sports (PCS)	$n = 30$ (Scienze motorie)
Dispositivo EEG	Natus XLTEK EEG 32 [®]
Elettrodi attivi	21 (sistema internazionale 10-20)
Riferimento durante acquisizione	Pz (elettrodo parietale centrale)
Elettrodo di terra	CPz
Frequenza di campionamento	256 Hz
Impedenza target durante acquisizione	$< 10 \text{ k}\Omega$ per tutti i canali
Trial IGT per soggetto	200 (2 blocchi da 100 trial ciascuno)
Durata media sessione	≈ 16 minuti
Campioni medi per soggetto	≈ 270336 campioni (≈ 1056 secondi)
Formato dati	CSV (EEG raw) + CSV (dati comportamentali IGT)
Standard organizzativo	BIDS-like (Brain Imaging Data Structure)

2.2 Descrizione dei gruppi sperimentali

I partecipanti appartengono a due gruppi: studenti di Ingegneria (ENG) e studenti di Scienze motorie e dello sport (PCS). La distinzione in gruppi è riportata nel file `participants.csv` tramite la colonna `GROUP`. La pipeline non discrimina tra gruppi: elabora tutti i soggetti allo stesso modo. L'analisi per gruppo è identificata come sviluppo futuro per investigare eventuali differenze nei pattern decisionali legati alla formazione accademica.

2.3 Protocollo sperimentale

La sessione sperimentale è strutturata in quattro momenti distinti con tempistiche standardizzate:

1. **Baseline (3 minuti):** il partecipante siede in posizione rilassata con gli occhi aperti, senza compiti da eseguire. Il segnale EEG viene registrato come riferimento basale per confronto con la fase task. Questa fase garantisce che i trial IGT inizino almeno 180 secondi dopo l'inizio della registrazione, rendendo tutti i 200 trial validi per l'estrazione dell'epoca $[-2s, 0s]$.
2. **Primo blocco IGT (100 decisioni):** il partecipante esegue il task scegliendo liberamente tra i quattro deck (A, B, C, D) visualizzati su schermo. In questa fase non è informato delle strutture di payoff e deve apprendere empiricamente le conseguenze delle proprie scelte.
3. **Pausa (4 minuti):** periodo di riposo inter-blocco per ridurre la fatica cognitiva e muscolare (artefatti EMG).
4. **Secondo blocco IGT (100 decisioni):** completamento del task. Il partecipante ha accumulato esperienza sulle conseguenze dei deck nel primo blocco.

Ogni singola iterazione del task è composta da due sotto-fasi cronologicamente distinte:

- **Fase pre-decisionale:** il partecipante delibera quale deck scegliere. La durata è variabile per ogni partecipante e ogni trial, dipendente dal *reaction time* (RT) individuale. Questa è la finestra di interesse per la nostra pipeline $[-2s, 0s]$.

- **Fase di feedback (1500 ms fissi):** il partecipante visualizza sul monitor il risultato della scelta (crediti vinti e crediti persi). In questa fase emergono i componenti ERP post-decisionali (FRN, P300) che il paper analizza in Fig. 9.

2.4 Iowa Gambling Task – Struttura dei payoff

L'Iowa Gambling Task è stato progettato originalmente da Bechara et al. (1994) come strumento clinico per valutare il processo decisionale in condizioni di incertezza, con applicazione iniziale nello studio di pazienti con lesioni della corteccia prefrontale ventromediale (vmPFC). Il task si basa sull'ipotesi del *somatic marker*: le scelte vantaggiose sono guidate da segnali somatici impliciti (stati corporei anticipatori) che il cervello associa alle conseguenze passate.

I quattro deck presentano strutture di payoff asimmetriche studiate per indurre preferenze a lungo termine diverse dalla preferenza immediata:

Table 2: Struttura completa dei payoff dei deck nell'IGT (Bechara et al., 1994)

Table 3: Struttura completa dei payoff dei deck nell'IGT (Bechara et al., 1994)

Deck	Guadagno fisso/carta	Perdita occasionale	Freq. perdita	Netto ogni 10 carte	Tipologia
A	+100	da 150 a 350 (variabile)	5/10	-250	Svantaggioso
B	+100	-1250 (fisso)	1/10	-250	Svantaggioso
C	+50	da 25 a 75 (variabile)	5/10	+250	Vantaggioso
D	+50	-250 (fisso)	1/10	+250	Vantaggioso

I deck A e B sono entrambi svantaggiosi a lungo termine (-250 ogni 10 carte) ma differiscono nella struttura delle perdite: A ha perdite frequenti e variabili (rischio distribuito), B ha una perdita rara ma molto elevata (rischio concentrato). Analogamente, C e D sono vantaggiosi (+250 ogni 10 carte) ma con struttura diversa: C ha perdite frequenti e moderate, D ha una perdita rara. Questa doppia asimmetria (deck A/C con perdite frequenti vs deck B/D con perdita rara) è rilevante per l'analisi del comportamento di risk aversion/risk seeking.

Nel dataset di Chávez-Sánchez et al. (2026), la Fig. 7 mostra l'evoluzione della proporzione di scelte vantaggiose $(C+D)/(A+B+C+D)$ nel corso dei 200 trial, evidenziando l'apprendimento progressivo dei partecipanti.

2.5 Struttura dei file del dataset

Il dataset segue una convenzione BIDS-like (*Brain Imaging Data Structure*). La pipeline implementata si aspetta la seguente struttura, con gestione automatica di varianti nei nomi dei file:

```

1 root/                                     # --dataset in run_pipeline.py
2   participants.csv                        # demografici: ID, sesso, eta, gruppo
3   obstetrics.csv                        # dati ostetrici (partecipanti femmine)
4   substance_use.csv                     # uso di sostanze (tutti i partecipanti)
5   dataset-description.csv                # metadati del dataset
6   s-01/
7     s-01_eeg.csv                         # EEG raw: righe=campioni, colonne=21 canali (uV)
8     s-01_igt.csv                         # dati comportamentali: TRIAL, DECK, EEG_SAMPLE, WIN,
9     s-01_preprocessed_eeg.csv            # EEG preprocessato dagli autori (opzionale, non
                                           usato)

```

```

10  s-02/
11    s-02_eeg.csv
12    s-02_igt.csv
13  ...
14  s-59/
15    s-59_eeg.csv
16    s-59_igt.csv

```

Listing 1: Struttura attesa della directory radice del dataset

Formato del file s-XX_eeg.csv

Il file EEG raw è una matrice di dimensioni ($N_{\text{campioni}} \times 21$), dove ogni colonna corrisponde a un elettrodo (nell'ordine definito da `CH_NAMES`) e ogni riga a un campione temporale. I valori sono in microvolt (μV). Per il soggetto s-01: $N_{\text{campioni}} = 270336$, corrispondente a $1056\text{ s} \approx 17.6\text{ min}$ a 256 Hz .

La prima riga contiene le intestazioni delle colonne con i nomi dei canali. La funzione `load_eeg_csv()` gestisce automaticamente file senza intestazione utilizzando i nomi di default da `CH_NAMES`.

Formato del file s-XX_igt.csv

Table 4: Colonne del file s-XX_igt.csv e corrispondenza nel codice

Colonna raw	Colonna normalizzata	Tipo	Descrizione
iteration / TRIAL	TRIAL	int	Numero del trial (1–200)
decision / DECK	DECK	str	Deck scelto: 'A', 'B', 'C', 'D'
EEG sample / EEG_SAMPLE	EEG_SAMPLE	int	Indice campione EEG della decisione
win / WIN	WIN	int	Crediti vinti in questo trial
lose / LOSS	LOSS	int	Crediti persi (valore negativo)
balance / NET	NET	int	Bilancio cumulativo
RT / RT	RT	float	Reaction time (millisecondi)

Nota – Robustezza nel parsing dei file IGT

La funzione `load_igt_csv()` normalizza i nomi delle colonne applicando: conversione in maiuscolo, rimozione degli spazi iniziali/finali, sostituzione degli spazi interni con underscore. La funzione `sync_eeg_igt()` ricerca la colonna EEG sample tramite pattern matching ("EEG" in col AND "SAMPLE" in col) per gestire varianti come "EEG SAMPLE", "EEG_SAMPLE", "eeg_sample". La colonna deck viene cercata in ordine prioritario tra: DECK, DECISION, CARD, CHOICE.

I file alternativi tentati automaticamente includono:

s-XX_eeg.csv → EEG.csv → eeg_raw.csv → raw_eeg.csv per l'EEG; s-XX_igt.csv → igt.csv → IGT.csv → behavioral.csv per il comportamentale.

2.6 Canali EEG e posizionamento sistema 10-20

I 21 elettrodi attivi sono definiti nel codice dalla costante globale `CH_NAMES`, identica in entrambi i moduli `eeg_preprocessing.py` e `eeg_ml_pipeline.py`:

```

1 CH_NAMES = [
2     "C3", "C4", "O1", "O2", "A1", "A2", "Cz",
3     "F3", "F4", "F7", "F8", "Fz",
4     "Fp1", "Fp2", "Fpz", "P3", "P4",
5     "T4", "T5", "T6", "Pz"
6 ]

```

Listing 2: Costante CH_NAMES (identica in entrambi i moduli)

Table 5: Elettrodi EEG, aree corticali corrispondenti e rilevanza specifica per l'IGT

Regione	Elettrodi	Rilevanza per l'IGT e pattern EEG attesi
Frontale	Fp1, Fp2, Fpz, F3, F4, F7, F8, Fz	Sede della corteccia prefrontale (vmPFC, dlPFC): FRN, DPN, theta frontale (working memory, monitoraggio rischio, controllo esecutivo)
Centrale	C3, C4, Cz	Corteccia motoria e premotoria: preparazione alla risposta motoria (pressione tasto), componente N200
Temporale	T4, T5, T6, A1, A2	Corteccia temporale: memoria episodica, elaborazione emotiva; A1/A2 sono auricolari (usati come riferimento in alcune analisi)
Parietale	P3, P4, Pz	Corteccia parietale: componente P300 (aggiornamento contesto, decisione), attenzione spaziale
Occipitale	O1, O2	Corteccia visiva: alpha resting-state (10 Hz), elaborazione visiva del feedback

Nota critica – Elettrodi auricolari A1 e A2

A1 e A2 sono elettrodi auricolari (lobi delle orecchie) usati tradizionalmente come riferimento nelle registrazioni cliniche. Nel dataset di Chávez-Sánchez et al. il riferimento è Pz, quindi A1 e A2 registrano attività fisiologica autentica. Tuttavia, l'analisi di feature importance con Random Forest ha mostrato che questi elettrodi figurano tra i più informativi, il che potrebbe riflettere: (1) attività muscolare residua temporale (artefatto EMG), (2) artefatti del riferimento Pz non completamente rimossi dal CAR, o (3) genuina attività temporale correlata alla decisione. La loro esclusione sistematica dalla feature matrix è identificata come sviluppo futuro da investigare.

3 Architettura Generale del Sistema

3.1 Panoramica dell'architettura a tre livelli

Il sistema è strutturato in tre livelli logici, ciascuno implementato come modulo Python autonomo con interfaccia pubblica ben definita (funzioni esportabili + argomenti da riga di comando via `argparse`):

Flusso dati completo dell'architettura

INPUT: s-XX_eeg.csv + s-XX_igt.csv per ogni soggetto (dataset Mendeley)
oppure: dati sintetici generati automaticamente da `generate_synthetic_data()`

`run_pipeline.py` → Orchestratore end-to-end

↓ **Step 0** (solo modalità `synthetic`):

`generate_synthetic_data()` → dati EEG simulati con struttura realistica

↓ **Step 1:** `eeg_preprocessing.py` – `run_full_pipeline()`

per ogni soggetto → `process_subject()`:

load CSV → MNE Raw → CAR → Bandpass FIR → Notch FIR → ICA → Z-score

→ sync EEG-IGT → epoche [−2s, 0s] → `.npy` per soggetto

↓ **Step 2:** `eeg_ml_pipeline.py` – `run_pipeline()`

PSD Welch → X ($N \times 105$) → LOSO CV → LR + SVM-RBF + RF

→ metriche aggregate → visualizzazioni → modelli `.joblib`

OUTPUT: `pipeline_output/` con CSV, PNG, `.joblib`, `pipeline_summary.csv`

3.2 Costanti globali e parametri configurabili

Entrambi i moduli principali definiscono costanti globali che centralizzano i parametri chiave dell'analisi. La tabella seguente riporta le costanti più rilevanti con i valori implementati nel codice:

Table 6: Costanti globali nei moduli Python con valori implementati

Costante	Valore	Tipo	Modulo	Descrizione
SFREQ	256	float	entrambi	Frequenza di campionamento (Hz)
N_CHANNELS	21	int	preproc	Numero canali EEG attivi
TMIN	−2.0	float	preproc	Inizio finestra epoche (secondi)
TMAX	0.0	float	preproc	Fine finestra epoche (secondi)
BASELINE	None	–	preproc	Nessuna baseline removal
BANDPASS_LOW	0.5	float	preproc	Taglio basso FIR (Hz)
BANDPASS_HIGH	70.0	float	preproc	Taglio alto FIR (Hz)
NOTCH_FREQ	50.0	float	preproc	Frequenza notch (Hz) – Europa
N_ICA_COMPONENTS	15	int	preproc	Componenti ICA FastICA
DECK_LABEL_MAP	A,B→0; C,D→1	dict	preproc	Mapping deck → etichetta binaria
LABEL_NAMES	0→disadv, 1→adv	dict	preproc	Nomi leggibili delle classi
RANDOM_SEED	42	int	ML	Seed globale per riproducibilità
FREQ_BANDS	5 bande	dict	ML	Dizionario bande PSD (Hz)
PALETTE	3 colori	list	ML	Colori per grafici (#1A56A0, ecc.)

3.3 Dipendenze software

Table 7: Dipendenze software complete della pipeline con versioni minime testate

Libreria	Versione min.	Utilizzo principale nel progetto
<code>numpy</code>	≥ 1.24	Array multidimensionali, pre-allocazione feature matrix, operazioni vettoriali
<code>scipy</code>	≥ 1.10	Calcolo PSD Welch (<code>scipy.signal.welch</code>), supporto filtraggio
<code>pandas</code>	≥ 2.0	Caricamento e normalizzazione CSV, DataFrame risultati, salvataggio
<code>mne</code>	≥ 1.6	Framework EEG: oggetti Raw, Epochs, ICA, filtri FIR, re-referencing
<code>scikit-learn</code>	≥ 1.3	Pipeline, StandardScaler, LR, SVM, RF, LOSO, GroupK-Fold, metriche
<code>joblib</code>	≥ 1.3	Parallelizzazione fold LOSO, serializzazione modelli <code>.joblib</code>
<code>matplotlib</code>	≥ 3.7	Figure diagnostiche preprocessing, grafici ML, configurazione stile
<code>seaborn</code>	≥ 0.12	Heatmap Confusion Matrix, violin/box plot, strip plot
<code>rich</code>	≥ 13.0	Output terminale premium: tabelle, pannelli, logging colorato
<code>pathlib</code>	<code>stdlib</code>	Gestione path cross-platform in tutti i moduli
<code>re</code>	<code>stdlib</code>	Pattern matching per scoperta soggetti (<code>s-\d+</code>)
<code>warnings</code>	<code>stdlib</code>	Soppressione <code>RuntimeWarning</code> durante ICA e filtraggio

3.4 Struttura dei moduli e funzioni esportate

Table 8: Funzioni pubbliche esportate da `eeg_preprocessing.py`

Funzione	Descrizione
<code>discover_subjects(root_dir)</code>	Scansiona la directory e restituisce lista ordinata di directory soggetto
<code>load_eeg_csv(subj_dir, sid)</code>	Carica EEG raw CSV, shape $(N_{\text{camp}}, 21)$, μV
<code>load_igt_csv(subj_dir, sid)</code>	Carica DataFrame IGT con normalizzazione nomi colonne
<code>build_mne_raw(data, names, sfreq)</code>	Crea MNE RawArray da array NumPy, converte $\mu\text{V} \rightarrow \text{V}$
<code>set_reference(raw, ref)</code>	Re-referencing CAR o canale specifico
<code>apply_filters(raw, l, h, notch)</code>	Bandpass FIR + notch FIR
<code>run_ica(raw, n_comp, seed)</code>	FastICA + rimozione EOG automatica, ritorna <code>(raw_clean, ica)</code>
<code>normalize_signal(raw)</code>	Z-score per canale sull'intera registrazione
<code>sync_eeg_igt(igt_df, n_samples)</code>	Allineamento EEG-IGT, validazione trial, generazione label
<code>extract_epochs(raw, sync_df)</code>	Estrazione epoche MNE $[-2\text{s}, 0\text{s}]$, ritorna <code>(epochs, labels, samples)</code>
<code>save_subject_output(sid, epochs, labels, samples, outdir)</code>	Salva <code>.npy</code> e <code>.npz</code> per soggetto
<code>build_dataset_dataframe(outdir)</code>	Aggrega output di tutti i soggetti in un unico DataFrame flat
<code>plot_raw_vs_clean(raw_pre, raw_post, sid, outdir)</code>	Figura comparativa pre/post preprocessing
<code>plot_psd(raw, sid, outdir, label)</code>	PSD media canali con marcatori filtri
<code>plot_epoch_mean(epochs, sid, outdir)</code>	ERP medio per classe su canali frontali
<code>process_subject(subj_dir, sid, outdir, figs)</code>	Pipeline completa per un singolo soggetto
<code>run_full_pipeline(root, outdir, figs, limit)</code>	Pipeline su tutti i soggetti, ritorna DataFrame riepilogo

Table 9: Funzioni pubbliche esportate da `eeg_ml_pipeline.py`

Funzione	Descrizione
<code>extract_psd_features(epoch, sfreq, bands, nperseg)</code>	PSD Welch per singola epoca, ritorna vettore ($n_{ch} \times n_{bande}$)
<code>get_feature_names(ch_names, bands)</code>	Lista nomi feature in formato <code>CH_banda</code>
<code>build_feature_matrix(epochs_data, sfreq, bands)</code>	Feature matrix completa ($N \times 105$), pre-allocata
<code>get_models(random_state)</code>	Dizionario {nome: Pipeline} con LR, SVM-RBF e RF
<code>_run_single_fold(fold_idx, ...)</code>	Helper per la parallelizzazione: un fold per tutti i modelli
<code>run_cross_validation(X, y, subj, models, cv, k, jobs)</code>	LOSO o GroupKFold parallelizzata, ritorna (<code>results</code> , <code>roc_data</code>)
<code>compute_summary_table(results, outdir)</code>	Media $\pm$ std per metrica e modello, stampa Rich e salva CSV
<code>plot_confusion_matrices(X, y, subj, models, outdir)</code>	CM aggregata LOSO, subplot 1×3 con annotazioni doppie
<code>plot_roc_curves(roc_data, outdir)</code>	ROC media $\pm$ std, tre modelli sovrapposti, griglia da 300 punti
<code>plot_feature_importance(X, y, names, outdir, top_n)</code>	Importanza RF (Gini) e coefficienti LR $ \beta $, top-20
<code>plot_per_fold_metrics(results, outdir)</code>	Violin plot ($N \geq 8$) oppure box plot delle metriche per fold
<code>save_models(X, y, models, outdir)</code>	Addestra su tutto il dataset e salva i modelli .joblib
<code>generate_synthetic_data(n_subj, ...)</code>	Dati EEG simulati con differenza di classe realistica
<code>load_preprocessed_data(input_dir)</code>	Carica tutti i file *_epochs.npy dalla directory di preprocessing
<code>run_pipeline(epochs, labels, subj, outdir, ...)</code>	Pipeline ML end-to-end in sei step

4 Pipeline di Preprocessing EEG (eeg_preprocessing.py)

4.1 Struttura del modulo e funzione principale

Il modulo `eeg_preprocessing.py` è organizzato in otto sezioni funzionali sequenziali, ciascuna implementata come insieme di funzioni autonome testabili indipendentemente. L'entry point principale per l'uso programmatico (chiamato dall'orchestratore) è `run_full_pipeline()`, che itera su tutti i soggetti invocando `process_subject()` per ognuno. L'entry point da riga di comando usa `argparse` con gli argomenti `-dataset`, `-output`, `-no-figures`, `-limit`.

Sequenza completa di preprocessing per soggetto – `process_subject()`

Step 1: `load_eeg_csv()` + `load_igt_csv()` $\rightarrow$ array NumPy + DataFrame

Step 2: `build_mne_raw()` $\rightarrow$ MNE RawArray (canali in V, shape $(21, N)$)

Step 3: `set_reference(ref="average")` $\rightarrow$ Common Average Reference (CAR)

Step 4: `apply_filters()` $\rightarrow$ bandpass FIR 0.5–70 Hz + notch FIR 50 Hz
(+ `plot_psd()` prima e dopo se `save_figures=True`)

Step 5: `run_ica()` $\rightarrow$ FastICA ($K = 15$) + rimozione EOG automatica

Step 6: `normalize_signal()` $\rightarrow$ z-score per canale sull'intera registrazione

```
(+ plot_raw_vs_clean() se save_figures=True)
Step 7: sync_eeg_igt() → validazione trial, label binarie, colonna VALID
Step 8: extract_epochs() → epoche MNE [-2s, 0s], allineamento labels
      (+ plot_epoch_mean() se save_figures=True)
Step 9: save_subject_output() → s-XX_epochs.npy, s-XX_labels.npy, ecc.
Return: dizionario stats con n_epochs, n_advantageous, n_disadvantageous, epoch_shape
```

L'intera sequenza è racchiusa in un blocco `try/except` che cattura qualsiasi eccezione, logga l'errore con traceback completo e restituisce `None` invece di interrompere l'elaborazione dell'intero dataset. Questo garantisce che un singolo soggetto corrotto non blocchi l'elaborazione degli altri.

4.2 Step 1 – Caricamento dei file sorgente

```
discover_subjects(root_dir)
```

La funzione scansiona la directory radice del dataset e restituisce la lista ordinata di percorsi alle directory soggetto che corrispondono al pattern regex `s-\d+`. L'ordinamento è alfabetico (quindi numerico per soggetti con ID a zero-padding). Se la directory non esiste, lancia `FileNotFoundError` con messaggio diagnostico completo.

Il pattern regex è applicato tramite `re.match(r"s-\d+", p.name)`, garantendo che directory con altri nomi (es. `participants.csv`, `README.md`) vengano ignorate.

```
load_eeg_csv(subject_dir, subject_id)
```

Carica il file EEG raw in formato CSV con `pandas.read_csv()` e restituisce un array NumPy di shape $(N_{\text{campioni}}, 21)$ con valori in μV . Il file atteso è `s-XX_eeg.csv`; se assente, vengono tentati in ordine: `eeg_raw.csv`, `EEG.csv`, `raw_eeg.csv`. Se nessun file è trovato, restituisce `None` con un warning nel log. In caso di errore durante la lettura (file corrotto, permessi), cattura l'eccezione e restituisce `None`.

```
load_igt_csv(subject_dir, subject_id)
```

Carica il file dei dati comportamentali IGT. La normalizzazione dei nomi delle colonne è applicata tramite:

```
1 df.columns = [c.strip().upper().replace(" ", "_") for c in df.columns]
```

Listing 3: Normalizzazione nomi colonne IGT

Questa trasformazione converte tutte le colonne in maiuscolo, rimuove spazi iniziali/finali e sostituisce spazi interni con underscore, rendendo il parsing robusto rispetto a varianti di formattazione tra soggetti o versioni del dataset. File alternativi tentati: `igt.csv`, `IGT.csv`, `behavioral.csv`.

4.3 Step 2 – Creazione oggetto MNE RawArray

```
build_mne_raw(eeg_data, ch_names, sfreq)
```

MNE-Python utilizza internamente i dati in Volt (non μV) e richiede la shape $(N_{\text{canali}}, N_{\text{campioni}})$, ovvero la trasposta rispetto alla convenzione dei file CSV. La conversione è:

Conversione unità di misura e orientamento per MNE

$$X_{\text{MNE}} = X_{\text{CSV}}^{\top} \times 10^{-6} \quad \text{shape: } (21, N_{\text{campioni}})$$

L'oggetto `mne.Info` viene creato con `mne.create_info()`, specificando i nomi dei canali, la frequenza di campionamento e il tipo di canale ("eeg"). Se il numero di canali nel file CSV differisce da `len(CH_NAMES)` (cioè da 21), la funzione emette un warning e utilizza nomi generici EEG000, EEG001, ... per evitare crash durante l'elaborazione.

```

1 # Conversione: (n_samples, n_ch) --> (n_ch, n_samples) in Volt
2 data_v = eeg_data.T * 1e-6
3
4 info = mne.create_info(ch_names=ch_names, sfreq=sfreq, ch_types="eeg")
5 raw = mne.io.RawArray(data_v, info, verbose=False)
6 logger.info(f" MNE RawArray: {raw.info['nchan']} canali, "
7             f"{raw.n_times} campioni, {raw.times[-1]:.1f}s")

```

Listing 4: Creazione oggetto MNE RawArray da array NumPy

4.4 Step 3 – Re-referencing: Common Average Reference

Il problema del riferimento EEG

Il segnale EEG è sempre misurato come differenza di potenziale tra l'elettrodo attivo e un elettrodo di riferimento fisico. Il dataset di Chávez-Sánchez et al. utilizza Pz come riferimento durante l'acquisizione. Questa scelta introduce un bias sistematico: il potenziale di Pz (che include attività parietale autentica, inclusa la componente P300 rilevante per l'IGT) viene sottratto da tutti i canali, potendo mascherare o amplificare componenti specifiche in diverse regioni.

Definizione matematica del CAR

Common Average Reference (CAR)

$$x_i^{\text{CAR}}(t) = x_i(t) - \frac{1}{N} \sum_{j=1}^N x_j(t) \quad \forall i = 1, \dots, N, \forall t$$

dove $N = 21$ è il numero di canali attivi e $x_i(t)$ è il potenziale del canale i al tempo t dopo la conversione in Volt.

Motivazioni della scelta CAR

1. **Riduzione artefatti di modo comune:** rumori presenti simultaneamente su tutti gli elettrodi (interferenza elettromagnetica ambientale, vibrazioni meccaniche trasmesse al soggetto, fluttuazioni dell'alimentazione del dispositivo) vengono cancellati dalla sottrazione della media comune, poiché compaiono identicamente in tutti i $x_j(t)$.
2. **Indipendenza dal sito di riferimento fisico:** il risultato del CAR non dipende dalla scelta di Pz e non introduce distorsioni sistematiche sulle bande frontali di interesse primario per l'IGT (Fz, F3, F4).
3. **Presupposto teorico:** con 21 elettrodi distribuiti uniformemente sul cuoio capelluto, la somma dei potenziali tende a zero per il teorema del potenziale sulla sfera. Con pochi elettrodi questo assunto è solo approssimativo, ma il CAR rimane la scelta più robusta disponibile.

4. **Standard nella letteratura EEG-based BCI:** il CAR è raccomandato per analisi task-related nelle review metodologiche EEG (Nunez et al., 1994).

```
1 raw_ref, _ = mne.set_eeg_reference(raw, ref_channels="average",
2                                   copy=True, verbose=False)
3 logger.info(" Re-referencing: Common Average Reference (CAR)")
```

Listing 5: Applicazione CAR tramite API MNE

Il parametro `copy=True` garantisce che l'oggetto `raw` originale non venga modificato in-place, preservando la possibilità di confronto pre/post per le figure diagnostiche.

4.5 Step 4 – Filtraggio Bandpass e Notch

Motivazione neurofisiologica del filtraggio bandpass

Il segnale EEG grezzo contiene componenti indesiderate che devono essere rimosse prima dell'analisi:

- **Sotto 0.5 Hz:** deriva DC (offset costante), fluttuazioni lente da movimenti del soggetto e artefatti da respirazione (≈ 0.2 Hz). Non hanno origine neurale e spostano arbitrariamente la baseline del segnale.
- **Sopra 70 Hz:** rumore elettronico del dispositivo, attività muscolare (EMG) ad alta frequenza (> 80 Hz). Il paper originale usa 70 Hz come limite superiore, preservando l'intera banda gamma.

Le bande neurofisiologicamente rilevanti preservate dal filtraggio 0.5–70 Hz sono:

Table 10: Bande di frequenza EEG, range, elettrodi chiave e ruolo specifico nel task IGT

Banda	Range (Hz)	Elettrodi chiave	Ruolo nel task IGT
Delta (δ)	1–4	Frontale, Parietale	Processi decisionali lenti, attenzione sostenuta, integrazione sensoriale
Theta (θ)	4–8	Fz, F3, F4, F7, F8	Working memory (aggiornamento), controllo cognitivo, monitoraggio rischio, componente FRN
Alpha (α)	8–13	O1, O2, P3, P4, Cz	Inibizione cognitiva, resting state, componente P300
Beta (β)	13–30	Frontale, Centrale	Elaborazione sensoriale, preparazione motoria alla risposta
Gamma (γ)	30–70	Distribuito	Binding cognitivo, integrazione multi-modale (componente N400)

La **banda theta frontale** (4–8 Hz, elettrodo Fz) è di particolare rilievo per questo task: Cui et al. (2013) dimostrano sperimentalmente che la potenza theta in Fz aumenta significativamente durante la fase pre-decisionale nell'IGT, riflettendo il carico della working memory e il monitoraggio del rischio. Questa evidenza giustifica sia la scelta della finestra temporale $[-2s, 0s]$ (che massimizza la cattura di questa attività) sia la priorità assegnata alla banda theta nell'analisi di feature importance.

Specifiche implementative del filtro bandpass

Parametri del filtro bandpass implementato in `apply_filters()`

$f_{\text{low}} = 0.5 \text{ Hz}$, $f_{\text{high}} = 70.0 \text{ Hz}$

Tipo: FIR (*Finite Impulse Response*), finestra di Hamming

Implementazione MNE:

```
raw.filter(l_freq=0.5, h_freq=70.0, method="fir", fir_window="hamming")
```

La scelta del filtro FIR (invece dell'IIR Butterworth 6° ordine usato nel paper originale) è motivata da:

- **Fase lineare garantita:** i filtri FIR con simmetria hanno risposta di fase lineare, che preserva la forma temporale dei transienti ERP. Questa proprietà è fondamentale per l'analisi delle componenti evento-correlate (FRN, P300, N400) previste come sviluppo futuro nella Sezione 19.
- **Stabilità assoluta:** i filtri FIR sono intrinsecamente stabili per qualsiasi scelta di parametri, a differenza dei filtri IIR che possono diventare instabili con ordine elevato.
- **Raccomandazione MNE:** la documentazione ufficiale MNE-Python raccomanda esplicitamente i filtri FIR per il preprocessing EEG destinato ad analisi di componenti evento-correlate.

```
1 raw_f = raw.copy()
2 raw_f.filter(
3     l_freq=l_freq, h_freq=h_freq,      # 0.5 Hz e 70.0 Hz
4     method="fir", fir_window="hamming",
5     verbose=False
6 )
7 logger.info(f" Bandpass FIR: {l_freq}--{h_freq} Hz")
```

Listing 6: Applicazione filtro bandpass FIR in `apply_filters()`

Filtro notch

```
1 raw_f.notch_filter(freqs=notch_freq, method="fir", verbose=False)
2 logger.info(f" Notch filter: {notch_freq} Hz")
3 # Valore di default: NOTCH_FREQ = 50.0 Hz (standard europeo)
4 # Per dataset messicano: cambiare NOTCH_FREQ = 60.0 nel codice
```

Listing 7: Applicazione filtro notch FIR

Nota critica – Frequenza di rete del dataset

Il dataset è stato acquisito all'Universidad de Guadalajara (Guadalajara, Jalisco, Messico), dove la rete elettrica opera a 60 Hz secondo lo standard ANSI/NEMA nordamericano. La pipeline usa `NOTCH_FREQ = 50.0 Hz` (standard europeo IEC). **Prima di elaborare il dataset reale**, verificare la PSD del segnale preprocessato di almeno un soggetto: se è presente un picco a 60 Hz non rimosso, modificare la costante `NOTCH_FREQ = 60.0` nel file `eeg_preprocessing.py` e ripetere l'intera pipeline. La funzione `plot_psd()` genera automaticamente un grafico che consente questa verifica visiva (linea verticale viola alla frequenza notch impostata).

4.6 Step 5 – Rimozione Artefatti con ICA

Il problema degli artefatti in EEG

Il segnale EEG registrato al cuoio capelluto è una sovrapposizione lineare di componenti provenienti da sorgenti diverse:

- **Attività neurale corticale:** la componente di interesse scientifico, con ampiezza tipica 5–20 μV ;
- **Artefatti oculari (EOG):** blink oculari con ampiezza 100–200 μV (10–20 volte superiore al segnale neurale), dominanti sui canali frontali Fp1, Fp2, Fpz; movimenti oculari saccadici e lenti;
- **Artefatti muscolari (EMG):** contrazioni dei muscoli facciali e cervicali, predominanti ad alte frequenze ($> 30 \text{ Hz}$), amplificate dallo stress del task;
- **Artefatti cardiaci (ECG):** battito cardiaco visibile come deflessione periodica distribuita su tutti i canali, con periodo $\approx 0.8\text{--}1.0 \text{ s}$.

L'ampiezza dei blink (100–200 μV) è la principale fonte di contaminazione: rende impossibile qualsiasi analisi affidabile dei canali frontali (Fp1, Fp2, Fpz) che sono proprio quelli più informativi per l'attività prefrontale nell'IGT.

Principio matematico dell'ICA

L'Independent Component Analysis (ICA) è un metodo di separazione cieca delle sorgenti (*Blind Source Separation*). Assume un modello generativo lineare stazionario:

Modello generativo ICA

$$\mathbf{X} = \mathbf{A} \mathbf{S}$$

- $\mathbf{X} \in \mathbb{R}^{N_{\text{ch}} \times T}$ – segnale EEG osservato ($N_{\text{ch}} = 21$ canali, T campioni)
- $\mathbf{S} \in \mathbb{R}^{K \times T}$ – sorgenti statisticamente indipendenti (K componenti)
- $\mathbf{A} \in \mathbb{R}^{N_{\text{ch}} \times K}$ – matrice di miscelazione (*mixing matrix*), incognita
- $\mathbf{W} = \mathbf{A}^{-1}$ – matrice di scomposizione (*unmixing matrix*), stimata dall'algoritmo

L'ICA stima $\mathbf{W}$ massimizzando la non-gaussianità (indipendenza statistica) delle sorgenti $\mathbf{S} = \mathbf{W}\mathbf{X}$.

Ricostruzione del segnale pulito (rimozione componenti artefattuali indicizzate da $\mathcal{E}$):

$$\hat{\mathbf{X}} = \mathbf{A}_{\mathcal{E}^c} \mathbf{S}_{\mathcal{E}^c} \quad (\text{colonne di } \mathbf{A} \text{ e righe di } \mathbf{S} \text{ non artefattuali})$$

Algoritmo FastICA – run_ica()

Viene utilizzato l'algoritmo FastICA implementato in MNE, che massimizza la non-gaussianità tramite la funzione di contrasto *logcosh*:

$$G(u) = \log \cosh(u), \quad g(u) = G'(u) = \tanh(u)$$

Table 11: Parametri FastICA implementati in `run_ica()`

Parametro MNE	Valore	Motivazione
<code>n_components</code>	15	15 comp. su 21 canali catturano > 95% varianza; meno componenti riducono frammentazione sorgenti genuine
<code>method</code>	"fastica"	Algoritmo iterativo efficiente; alternativa "infomax" disponibile ma più lenta
<code>max_iter</code>	3000	Aumentato da 1500 per ridurre <code>ConvergenceWarning</code> ; il costo computazionale extra è trascurabile
<code>fit_params["tol"]</code>	10^{-3}	Tolleranza leggermente più larga rispetto a 10^{-4} ; migliora robustezza su dati sintetici con rumore elevato
<code>random_state</code>	42	Seed fisso per riproducibilità completa tra esecuzioni

Pre-filtraggio a 1 Hz per la stima ICA

Secondo la documentazione MNE, l'ICA deve essere fittata su una copia del segnale pre-filtrata con taglio basso a 1 Hz (non 0.5 Hz). La deriva lenta sotto 1 Hz introduce non-stazionarietà che può compromettere la convergenza e la qualità della decomposizione ICA.

```

1 # Copia con pre-filtro a 1 Hz (raccomandazione MNE per ICA)
2 raw_hp = raw.copy().filter(l_freq=1.0, h_freq=None, verbose=False)
3 ica.fit(raw_hp, verbose=False)

```

Listing 8: Pre-filtraggio interno a 1 Hz per fitting ICA

Il pre-filtro viene applicato *solo* per la fase di fitting dell'ICA e *non* modifica il segnale principale (che rimane filtrato a 0.5 Hz). La rimozione delle componenti artefattuali viene poi applicata al segnale principale 0.5 Hz.

Identificazione automatica delle componenti artefattuali

Le componenti ICA associate ad artefatti oculari vengono identificate automaticamente tramite il metodo `find_bads_eog()` di MNE, che calcola la correlazione di Pearson tra ogni componente ICA e i canali frontali utilizzati come proxy EOG (Fp1, Fp2, Fpz):

```

1 # Usa i canali frontali presenti nel set come proxy EOG
2 eog_channels = [ch for ch in ["Fp1", "Fp2", "Fpz"]
3                  if ch in raw.ch_names]
4
5 if eog_channels:
6     eog_indices, eog_scores = ica.find_bads_eog(
7         raw, ch_name=eog_channels,
8         threshold=3.0,      # soglia: 3 sigma sopra la media delle correlazioni
9         verbose=False
10    )
11     ica.exclude = eog_indices
12     logger.info(f" Componenti EOG escluse: {eog_indices}")
13 else:
14     logger.warning(" Nessun canale EOG frontale trovato.")
15
16 # Applica ICA: ricostruzione senza componenti escluse

```

```

17 raw_clean = raw.copy()
18 ica.apply(raw_clean, verbose=False)

```

Listing 9: Identificazione e rimozione componenti EOG tramite ICA

Il parametro `threshold=3.0` esclude le componenti la cui correlazione con il canale EOG proxy supera 3 deviazioni standard rispetto alla distribuzione delle correlazioni di tutte le $K = 15$ componenti. Per il soggetto `s-01` del dataset reale, le componenti indice 0 e 1 vengono tipicamente identificate come artefatti oculari: la componente 0 corrisponde ai blink (attivazione simmetrica su Fp1 e Fp2), la componente 1 ai movimenti oculari orizzontali (attivazione asimmetrica Fp1 vs Fp2).

4.7 Step 6 – Normalizzazione Z-Score per Canale

Motivazione della normalizzazione inter-soggetto

L'ampiezza assoluta del segnale EEG varia considerevolmente tra soggetti diversi a causa di fattori anatomici e strumentali:

- **Anatomia individuale:** diverso spessore del cuoio capelluto, del cranio osseo e del fluido cerebrospinale tra i canali del segnale sorgente e gli elettrodi; questa variabilità produce differenze di ampiezza sistematiche tra soggetti.
- **Impedenza degli elettrodi:** variazione dell'impedenza di contatto da sessione a sessione (posizionamento, umidità del gel, preparazione della cute).
- **Attività cerebrale basale:** differenze individuali nell'ampiezza dell'oscillazione alpha dominante (frequenza individuale alpha, IAF), che costituisce il ritmo di fondo variabile tra soggetti.

Senza normalizzazione, un classificatore addestrato su più soggetti tende a imparare le differenze di ampiezza individuali (caratteristiche identitarie del soggetto) piuttosto che i pattern decisionali genuini – un meccanismo di data leakage implicito.

Formula e implementazione

Normalizzazione Z-Score per canale – `normalize_signal()`

$$z_i(t) = \frac{x_i(t) - \mu_i}{\sigma_i} \quad \text{per ogni canale } i = 1, \dots, 21$$

dove:

$$\mu_i = \frac{1}{T} \sum_{t=1}^T x_i(t), \quad \sigma_i = \sqrt{\frac{1}{T} \sum_{t=1}^T (x_i(t) - \mu_i)^2}$$

La normalizzazione è applicata sull'intera registrazione (non per singola epoca) per preservare la struttura temporale relativa del segnale.

```

1 raw_n = raw.copy()
2 data = raw_n.get_data() # shape: (21, N_campioni)
3 mu = data.mean(axis=1, keepdims=True)
4 sigma = data.std(axis=1, keepdims=True)
5 sigma[sigma < 1e-10] = 1e-10 # protezione divisione per zero
6 raw_n._data = (data - mu) / sigma
7 logger.info(" Normalizzazione z-score per canale applicata")

```

Listing 10: Normalizzazione z-score per canale sull'intera registrazione

La soglia $1e-10$ protegge dalla divisione per zero che potrebbe verificarsi con canali piatti (es. elettrodo disconnesso) su dati sintetici. Il risultato ha media zero e deviazione standard unitaria per ogni canale.

5 Sincronizzazione EEG-IGT ed Estrazione Epoche

5.1 Meccanismo di sincronizzazione – `sync_eeg_igt()`

Il file `s-XX_igt.csv` contiene la colonna `EEG_SAMPLE` che specifica l'indice esatto del campione EEG corrispondente al momento in cui il partecipante ha effettuato la scelta (pressione del tasto). Questa colonna è stata registrata in tempo reale durante l'esperimento tramite il software di acquisizione Natus XLTEK, garantendo una sincronizzazione precisa a livello di singolo campione ($\Delta t = 1/256 \text{ Hz} \approx 3.9 \text{ ms}$).

La funzione `sync_eeg_igt()` esegue i seguenti passi:

1. Identifica la colonna EEG sample tramite pattern matching ("`EEG`" in `col` AND "`SAMPLE`" in `col`);
2. Converte i campioni in secondi: $t_{\text{dec}}^{(k)} = \text{EEG_SAMPLE}^{(k)} / f_s$ dove $f_s = 256 \text{ Hz}$;
3. Identifica la colonna deck tramite ricerca prioritaria (`DECK` → `DECISION` → `CARD` → `CHOICE`);
4. Mappa il deck sulla label binaria tramite `DECK_LABEL_MAP`;
5. Gestisce deck non riconosciuti: drop dei trial con warning che riporta i valori sconosciuti;
6. Calcola la colonna `VALID` per la validazione della finestra temporale.

5.2 Validazione dei trial

Un trial è considerato valido per l'estrazione dell'epoca se e solo se l'intera finestra $[-2s, 0s]$ ricade all'interno della registrazione EEG disponibile:

$$\text{VALID}^{(k)} = (\text{EEG_SAMPLE}^{(k)} \geq n_{\text{pre}}) \quad \text{dove} \quad n_{\text{pre}} = |\text{TMIN}| \times f_s = 2 \times 256 = 512$$

Nel dataset reale tutti i 200 trial risultano validi per ogni soggetto, poiché la baseline iniziale di 3 minuti (768 s $\gg$ 512 campioni) garantisce un ampio margine prima del primo trial IGT. Il conteggio di trial validi è riportato nel log con distribuzione delle classi.

5.3 Generazione delle label binarie

Mapping deck – label binaria (`DECK_LABEL_MAP`)

$$\text{label}^{(k)} = \begin{cases} 0 & \text{se } \text{deck}^{(k)} \in \{A, B\} \quad (\text{disadvantageous} - \text{svantaggioso}) \\ 1 & \text{se } \text{deck}^{(k)} \in \{C, D\} \quad (\text{advantageous} - \text{vantaggioso}) \end{cases}$$

La motivazione neuroscientifica di questo raggruppamento è: la distinzione clinicamente rilevante nell'IGT è tra scelte impulsive a guadagno immediato (A e B, con perdita netta a lungo termine) e scelte ottimali (C e D, con guadagno netto a lungo termine). Questa dicotomia è supportata dall'ipotesi del somatic marker (Bechara et al., 1994) e dalla Fig. 7 del paper che usa questa dicotomia come metrica di performance comportamentale.

5.4 Estrazione delle epoche – `extract_epochs()`

La funzione costruisce l'array di eventi MNE nel formato ($N_{\text{trial}} \times 3$): ogni riga contiene [`campione`, 0, `event_id`], dove `event_id`=1 per la classe disadvantageous e `event_id`=2 per la classe advantageous. Il costruttore `mne.Epochs` estrae la finestra $[TMIN, TMAX] = [-2s, 0s]$:

Shape del tensore di output per soggetto

$$\mathbf{X}_{\text{subj}} \in \mathbb{R}^{200 \times 21 \times 513} \quad \mathbf{y}_{\text{subj}} \in \{0, 1\}^{200}$$

200 trial $\times$ 21 canali $\times$ 513 campioni ($= 2s \times 256 \text{ Hz} + 1 \text{ per } t = 0$)

MNE include il campione corrispondente a $t = 0$ (il momento esatto della decisione), producendo 513 campioni effettivi invece di 512. Il parametro `event_repeated="drop"` elimina trial con lo stesso indice campione (caso raro ma gestito). Il parametro `baseline=None` disabilita la baseline correction (motivazione in Sezione 18).

Allineamento labels con `epochs.selection`

MNE può scartare internamente alcune epoche a causa di: eventi ripetuti con `event_repeated="drop"`, trial che cadono ai bordi della registrazione, o altri controlli interni. È fondamentale allineare i vettori `labels` e `sample_indices` alle epoche effettivamente mantenute:

```
1 # epochs.selection: array degli indici degli eventi originali mantenuti
2 if hasattr(epochs, "selection") and len(epochs.selection) < len(labels):
3     labels = labels[epochs.selection]
4     sample_indices = sample_indices[epochs.selection]
5     logger.info(f" MNE ha scartato "
6                 f"{len_orig - len(labels)} epoche (bordi/duplicati)")
```

Listing 11: Allineamento labels con indici MNE `epochs.selection`

Motivazione neuroscientifica della finestra $[-2s, 0s]$

La scelta della finestra pre-decisionale è motivata da evidenze convergenti dalla letteratura neuroscientifica:

- **Decision Preceding Negativity (DPN):** Bianchin e Angrilli (2011) identificano una negatività frontale lenta a Fz che inizia ≈ 500 ms prima del momento della risposta motoria nell'IGT. Questo componente riflette il processo di valutazione comparativa del rischio tra i deck, ed è presente nella finestra $[-2s, 0s]$.
- **Slow Cortical Potentials (SCP):** potenziali corticali lenti che si sviluppano su scale temporali di 1–2s, associati alla preparazione cognitiva e motoria. Richiedono una finestra temporale ampia per essere catturati.
- **Attività theta frontale:** l'oscillazione theta (4–8 Hz) in Fz è massima nei 1.5–2s che precedono la scelta nell'IGT (Cui et al., 2013), riflettendo il carico della working memory durante la deliberazione.
- **Esclusione del feedback:** la finestra $[-2s, 0s]$ non include il periodo di feedback ($t > 0$, durata fissa 1500 ms), evitando la contaminazione con i componenti post-decisionali FRN (Feedback-Related Negativity, picco $\approx +250$ ms) e P300 (picco $\approx +400$ ms) che riflettono l'elaborazione del risultato piuttosto che la deliberazione.

6 Output del Preprocessing e Formato per ML

6.1 File salvati per soggetto – `save_subject_output()`

Per ogni soggetto elaborato con successo vengono salvati quattro file nella directory `preprocessing/epochs/`:

Table 12: File di output per soggetto nella directory `preprocessing/epochs/`

File	Shape / Formato	Contenuto
<code>s-XX_epochs.npy</code>	(200, 21, 513) float64	Segnale EEG z-normalizzato per ogni trial
<code>s-XX_labels.npy</code>	(200,) int	Label binarie: 0=svantaggioso, 1=vantaggioso
<code>s-XX_samples.npy</code>	(200,) int	Indici campione EEG originali (colonna <code>EEG_SAMPLE</code>)
<code>s-XX_info.npz</code>	array strutturato	Metadati: <code>ch_names</code> , <code>sfreq</code> , <code>tmin</code> , <code>tmax</code>

Il formato NumPy `.npy` (binario, single-array) è scelto per efficienza di I/O e compatibilità diretta con qualsiasi framework ML (scikit-learn, PyTorch, TensorFlow). Il file `.npz` (multi-array compresso) per i metadati garantisce la persistenza di tutte le informazioni necessarie per ricaricare correttamente i dati in sessioni future.

6.2 File riepilogativo di pipeline – `pipeline_summary.csv`

Al termine dell'elaborazione di tutti i soggetti, `run_full_pipeline()` produce il file `pipeline_summary.csv` nella directory di output principale. Il file contiene una riga per ogni soggetto elaborato con successo, con le colonne:

Table 13: Colonne del file `pipeline_summary.csv`

Colonna	Tipo	Descrizione
<code>subject_id</code>	str	Identificativo soggetto (es. <code>s-01</code>)
<code>n_trials_total</code>	int	Trial totali nel file IGT
<code>n_epochs</code>	int	Epoche estratte con successo
<code>n_advantageous</code>	int	Trial con label 1 (C/D)
<code>n_disadvantageous</code>	int	Trial con label 0 (A/B)
<code>epoch_shape</code>	tuple	Shape del tensore (<code>n_epochs</code> , 21, 513)

6.3 Caricamento per classificazione

Il modulo `eeg_ml_pipeline.py` carica automaticamente l'output del preprocessing tramite `load_preprocessed_data()`. Per utilizzo diretto da notebook Jupyter o script esterni:

```

1 import numpy as np
2 from pathlib import Path
3
4 epochs_dir = Path("./preprocessing/epochs") # output del preprocessing
5
6 all_X, all_y, all_subjects = [], [], []
7 for label_file in sorted(epochs_dir.glob("*_labels.npy")):
8     sid = label_file.stem.replace("_labels", "") # es. "s-01"
9     epochs = np.load(epochs_dir / f"{sid}_epochs.npy") # (200, 21, 513)
10    labels = np.load(label_file) # (200,)
11    all_X.append(epochs)
12    all_y.append(labels)
13    all_subjects.extend([sid] * len(labels))
14
```

```

15 X      = np.concatenate(all_X, axis=0)      # (N_tot, 21, 513)
16 y      = np.concatenate(all_y, axis=0)      # (N_tot,)
17 subjects = np.array(all_subjects)           # per LOSO CV
18 # N_tot = 55 soggetti * 200 trial = 11.000

```

Listing 12: Caricamento manuale del dataset completo per ML da notebook

6.4 Funzione `build_dataset_dataframe()`

La funzione `build_dataset_dataframe()` aggrega gli output di tutti i soggetti in un unico `pd.DataFrame` con layout flat (una riga per trial, colonne = feature piattizzate). Il formato flat ha la shape $(N_{\text{tot}}, 4 + 21 \times 513)$ dove le 4 colonne iniziali sono metadati (`subject_id`, `epoch_idx`, `label`, `label_name`) e le restanti sono i valori EEG piattizzati (`f00000`, `f00001`, ...).

Nota – Uso del formato flat

Il formato flat (`build_dataset_dataframe()`) è comodo per analisi esplorative con `pandas` ma sconsigliato per dataset grandi: $55 \times 200 \times 21 \times 513 = 118.6 \text{ M float64} \approx 950 \text{ MB}$ in memoria. Per la pipeline ML si raccomanda il formato `.npy` per soggetto + caricamento tramite `load_preprocessed_data()`, che gestisce efficientemente la memoria.

7 Funzioni di Visualizzazione Diagnostica

Le tre funzioni di visualizzazione diagnostica del preprocessing sono richiamate automaticamente all'interno di `process_subject()` quando il parametro `save_figures=True` (default). I file PNG vengono salvati nella directory `preprocessing/figures/` con DPI 120 (qualità sufficiente per revisione visiva, dimensione file contenuta).

7.1 `plot_raw_vs_clean()`

Genera una figura con $|\text{canali selezionati}| \times 2$ subplot, mostrando i primi `duration=5.0` secondi di segnale per i canali Fz, Cz, Pz, F3, F4, O1 (quelli disponibili in `CH_NAMES`), prima e dopo l'intero preprocessing. I canali sono selezionati come `[c for c in channels_to_plot if c in raw_before.ch_names][:6]`. Il segnale è mostrato in μV (convertito da V moltiplicando per 10^6). La colonna sinistra mostra il segnale grezzo (colore `steelblue`), la colonna destra il segnale preprocessato (`darkorange`).

Questo confronto visivo è fondamentale per verificare:

- L'efficacia del filtraggio bandpass nella rimozione della deriva DC;
- L'efficacia dell'ICA nella rimozione dei blink oculari (visibili come spike di ampiezza elevata su Fp1, Fp2, Fpz);
- L'effetto della normalizzazione z-score sull'ampiezza complessiva.

Salvato come `s-XX_raw_vs_clean.png`.

7.2 `plot_psd()`

Calcola e visualizza la PSD media su tutti i canali (metodo di Welch con `nperseg=SFREQ*2`). L'asse Y è in scala logaritmica (`semilogy`) per visualizzare efficacemente il range dinamico del segnale EEG (tipicamente 10^3 – $10^6 \mu\text{V}^2/\text{Hz}$). Tre linee verticali tratteggiate indicano:

- Rossa: taglio basso bandpass (0.5 Hz);
- Verde: taglio alto bandpass (70.0 Hz);

- Viola: frequenza notch (50.0 Hz).

La funzione viene chiamata due volte per ogni soggetto: prima del filtraggio (`label="pre_filter"`) e dopo (`label="post_filter"`), producendo i file `s-XX_psd_pre_filter.png` e `s-XX_psd_post_filter.png`. Il confronto permette di verificare la corretta applicazione dei filtri e l'eventuale presenza di picco residuo a 60 Hz non rimosso.

7.3 `plot_epoch_mean()`

Visualizza il segnale medio delle epoche per classe (advantageous vs disadvantageous) per i canali frontali chiave: Fz, Cz, F3, F4 (quelli disponibili in `CH_NAMES`). Per ogni canale e classe viene mostrata la media con banda di errore ± 1 SEM (Standard Error of the Mean): $SEM = \sigma / \sqrt{n}$. La linea verticale a $t = 0$ indica il momento della decisione. Il segnale è mostrato in μV .

Nota – Interpretazione del plot ERP medio

Questa visualizzazione è puramente diagnostica a livello di singolo soggetto: non costituisce un'analisi ERP formale, che richiederebbe averaging su molti soggetti per emergere dal rumore di fondo. Le differenze visibili tra le due classi (se presenti) indicano la presenza di componenti decisionali nella finestra pre-decisionale per quel soggetto specifico, ma non sono statisticamente significative a livello individuale. L'analisi ERP formale (replicazione Fig. 9 del paper) è identificata come sviluppo futuro.

Salvato come `s-XX_erp_mean.png`.

8 Pipeline di Feature Extraction PSD (`eeg_ml_pipeline.py`)

8.1 Panoramica e motivazione della scelta PSD

Il modulo `eeg_ml_pipeline.py` riceve in input le epoche EEG preprocessate dalla Parte I (tensori `.npy` di shape (200, 21, 513) per soggetto) e produce modelli di classificazione addestrati e valutati con metodologia cross-subject rigorosa.

La scelta della Power Spectral Density (PSD) come feature principale è motivata da una serie di considerazioni statistico-metodologiche rispetto alle alternative disponibili:

- **Singolo trial vs averaging:** le feature ERP (FRN, P300, N400) identificate dal paper emergono affidabilmente solo dopo mediazione su decine o centinaia di trial dello stesso soggetto. Con ≈ 200 trial per soggetto (suddivisi in due classi $\approx 100/100$), la stima ERP di singolo trial ha un rapporto segnale/rumore insufficiente per un classificatore cross-subject generalizzabile. La PSD, stimata su una finestra di 2s, produce feature stabili anche a livello di singolo trial.
- **Robustezza al rumore:** il metodo di Welch (media di periodogrammi su segmenti sovrapposti) riduce la varianza della stima PSD rispetto alla FFT diretta di un fattore $\approx K$ (numero di segmenti), producendo feature più stabili.
- **Coerenza con la letteratura EEG-IGT:** Aram et al. (2019) e Balconi & Angioletti (2022) identificano nella potenza spettrale per banda – in particolare theta frontale e alpha occipitale – i correlati neurali più informativi per la distinzione advantageous/disadvantageous nell'IGT.
- **Corrispondenza con gli ERP del paper:** la potenza theta in Fz (4–8 Hz) è il correlato spettrale della componente FRN (*Feedback-Related Negativity*) identificata dal paper in Fig. 9a. Il fatto che la FRN abbia la sua frequenza dominante nella banda theta giustifica

l'uso della PSD theta come proxy dell'attività frontale pre-decisionale.

- **Interpretabilità:** ogni feature PSD ha un chiaro significato neurofisiologico (potenza della banda X nell'elettrodo Y), facilitando l'interpretazione clinica dei risultati.

8.2 Algoritmo di Welch – `extract_psd_features()`

Principio matematico

Il metodo di Welch (1967) stima la Power Spectral Density dividendo il segnale in K segmenti sovrapposti di lunghezza L , calcolando il periodogramma modificato (con finestra) per ciascun segmento e mediando i risultati:

Stima PSD – Metodo di Welch

$$\hat{P}(f) = \frac{1}{K} \sum_{k=1}^K \frac{|\text{DFT}[w \cdot x_k](f)|^2}{f_s \cdot \sum_{n=0}^{L-1} w^2(n)}$$

dove:

- K = numero di segmenti sovrapposti (overlap 50% di default)
- w = finestra di Hann: $w(n) = 0.5(1 - \cos(2\pi n/(L-1)))$
- $f_s = 256$ Hz, $L = \text{nperseg} = 256$ campioni (= 1 s)
- $\sum_n w^2(n)$: fattore di normalizzazione per la perdita di energia della finestra

Potenza media nella banda $[f_{\text{low}}, f_{\text{high}}]$:

$$P_{\text{banda}} = \frac{1}{|F|} \sum_{f \in F} \hat{P}(f), \quad F = \{f : f_{\text{low}} \leq f < f_{\text{high}}\}$$

Con $\text{nperseg} = 256$ e $f_s = 256$ Hz, la risoluzione frequenziale è:

$$\Delta f = \frac{f_s}{\text{nperseg}} = \frac{256}{256} = 1 \text{ Hz}$$

Questo significa che ogni bin di frequenza corrisponde esattamente a 1 Hz, garantendo un'assegnazione netta dei bin alle bande di frequenza definite in `FREQ_BANDS`.

Implementazione dettagliata

```

1 def extract_psd_features(epoch, sfreq=256, freq_bands=FREQ_BANDS,
2                           nperseg=None):
3     """
4     Estrae feature PSD (Welch) per singola epoca EEG.
5
6     Parameters
7     -----
8     epoch      : np.ndarray (n_channels, n_samples)
9                  Singola epoca EEG preprocessata
10    sfreq       : float, frequenza di campionamento (Hz)
11    freq_bands  : dict, {nome: (f_low, f_high)} -- bande di frequenza
12    nperseg     : int or None, lunghezza segmento Welch
13                  (default: min(256, n_samples) = finestra 1 s)
14
15    Returns
16    -----

```

```

17     features      : np.ndarray (n_channels * n_bands,) dtype=float32
18                     Vettore feature: potenza media per banda per canale
19     """
20     n_channels, n_samples = epoch.shape
21     if nperseg is None:
22         nperseg = min(256, n_samples)    # 1s a 256 Hz
23
24     features = []
25     for ch_idx in range(n_channels):
26         # Welch PSD per il singolo canale
27         freqs, psd = welch(epoch[ch_idx], fs=sfreq,
28                             nperseg=nperseg, window="hann")
29         for band_name, (low, high) in freq_bands.items():
30             band_mask = (freqs >= low) & (freqs < high)
31             band_power = np.mean(psd[band_mask]) if band_mask.sum() > 0 else
32                             0.0
33             features.append(band_power)
34
35     return np.array(features, dtype=np.float32)

```

Listing 13: Implementazione completa di `extract_psd_features()`

La condizione `if mask.sum() > 0 else 0.0` gestisce il caso limite in cui nessun bin di frequenza ricade nella banda specificata (possibile con `nperseg` molto piccolo su dati sintetici).

Dizionario `FREQ_BANDS`

```

1 FREQ_BANDS = {
2     "delta": (0.5, 4.0),    # Processi decisionali lenti, SCP
3     "theta": (4.0, 8.0),   # Working memory, FRN, monitoraggio rischio
4     "alpha": (8.0, 13.0),  # Inibizione cognitiva, P300, resting state
5     "beta": (13.0, 30.0),  # Preparazione motoria, elaborazione sensoriale
6     "gamma": (30.0, 45.0), # Binding cognitivo, N400 (limite 45 non 70)
7 }

```

Listing 14: Costante `FREQ_BANDS` definita in `eeg_ml_pipeline.py`

Table 14: Bande di frequenza estratte: definizioni, correlati neurali nell'IGT e nota sul limite gamma

Banda	Range (Hz)	Elettrodi chiave	Rilevanza specifica nell'IGT
delta (δ)	0.5–4	Frontale, Parietale	Attenzione sostenuta, processi decisionali lenti; Slow Cortical Potentials (SCP) nella finestra $[-2s, 0s]$
theta (θ)	4–8	Fz, F3, F4, F7, F8	Working memory (aggiornamento buffer delle scelte precedenti), controllo cognitivo ACC, monitoraggio rischio, componente FRN
alpha (α)	8–13	O1, O2, P3, P4	Inibizione cognitiva delle regioni non coinvolte, resting state marker, componente P300
beta (β)	13–30	Frontale, C3, C4	Elaborazione sensoriale visiva (visualizzazione deck), preparazione alla risposta motoria
gamma (γ)	30–45	Distribuito	Binding cognitivo, integrazione multi-modale, correlato della componente N400

Nota – Limite superiore gamma a 45 Hz (non 70 Hz)

Il paper originale usa un taglio alto a 70 Hz per il filtraggio bandpass, preservando tutta l'attività gamma. Tuttavia, nella feature extraction la banda gamma viene estratta fino a 45 Hz (non 70 Hz) per il seguente motivo: al di sopra di 45 Hz, l'attività EMG muscolare residua (contrazioni facciali e cervicali non completamente rimosse dall'ICA) domina sul segnale neurale gamma, introducendo feature spurie non legate all'attività cognitiva. Il range 30–45 Hz è lo standard consolidato nella letteratura per la banda gamma nei task cognitivi con dataset EEG non-invasivo.

8.3 Costruzione della feature matrix – build_feature_matrix()

La funzione `build_feature_matrix()` applica `extract_psd_features()` a ogni epoca di ogni soggetto, costruendo la feature matrix completa. L'implementazione pre-alloca l'array per evitare la concatenazione iterata (`np.append` in loop è $O(N^2)$):

```

1 def build_feature_matrix(epochs_data, sfreq=SFREQ, freq_bands=FREQ_BANDS):
2     """
3     Parameters
4     -----
5     epochs_data : list of np.ndarray, ciascuno di shape (n_epochs, n_ch, n_t)
6     Returns
7     -----
8     X : np.ndarray (N_tot, n_ch * n_bands) -- feature matrix
9     """
10    n_features = len(CH_NAMES) * len(freq_bands) # 21 * 5 = 105
11    N_total    = sum(e.shape[0] for e in epochs_data)
12
13    # Pre-allocazione: evita concatenazione iterata O(N^2)

```

```

14 X = np.zeros((N_total, n_features), dtype=np.float32)
15
16 row = 0
17 for subj_epochs in epochs_data:
18     n_ep = subj_epochs.shape[0]
19     for i in range(n_ep):
20         X[row] = extract_psd_features(subj_epochs[i], sfreq, freq_bands)
21         row += 1
22 return X

```

Listing 15: Costruzione efficiente della feature matrix con pre-allocazione

8.4 Struttura e nomi delle feature – get_feature_names()

Struttura del vettore feature per singola epoca

L'ordine delle feature segue il doppio loop canale (loop esterno) × banda (loop interno):

$$\mathbf{x} = \underbrace{[x_{C3,\delta}, x_{C3,\theta}, x_{C3,\alpha}, x_{C3,\beta}, x_{C3,\gamma}]}_{\text{CH: C3 - 5 feature}} \underbrace{[x_{C4,\delta}, \dots, x_{C4,\gamma}]}_{\text{CH: C4}} \dots \underbrace{[x_{Pz,\delta}, \dots, x_{Pz,\gamma}]}_{\text{CH: Pz}}$$

Dimensione totale: 21 canali × 5 bande = **105** feature per epoca.

Naming convention: CH_banda – es. Fz_theta, O1_alpha, Fp1_delta, C3_gamma.

Feature matrix completa:

$$\mathbf{X} \in \mathbb{R}^{n_{\text{epochs}} \times 105}, \quad n_{\text{epochs}} = 55 \text{ soggetti} \times 200 \text{ trial} = 11.000$$

```

1 def get_feature_names(ch_names=CH_NAMES, freq_bands=FREQ_BANDS):
2     """Ritorna lista di 105 nomi: ['C3_delta', 'C3_theta', ...,
3     'Pz_gamma']"""
4     return [f"{ch}_{band}" for ch in ch_names for band in freq_bands.keys()]
5     # Ordine identico al loop in extract_psd_features():
6     # for ch in range(n_channels):          <-- loop esterno
7     #     for band_name, ... :                <-- loop interno

```

Listing 16: Generazione lista nomi feature – get_feature_names()

I nomi delle feature vengono salvati in `feature_names.csv` all'interno della directory di output, insieme a `feature_matrix.npy` (la matrice X completa) per consentire analisi di interpretabilità post-hoc con strumenti esterni (es. SHAP, LIME).

9 Definizione dei Modelli di Machine Learning

I tre classificatori implementati in `get_models()` sono scelti per caratteristiche complementari e crescente complessità computazionale: lineare/interpretabile (LR), kernel non-lineare (SVM-RBF), ensemble non-parametrico (RF). Tutti sono incapsulati in una `sklearn.Pipeline` con `StandardScaler` obbligatorio come primo step.

9.1 Logistic Regression – baseline lineare

La Logistic Regression costituisce il baseline interpretabile della pipeline. Modella la probabilità a posteriori della classe positiva come funzione sigmoide lineare:

Logistic Regression con regolarizzazione L2

$$P(y = 1 \mid \mathbf{x}) = \sigma(\beta_0 + \boldsymbol{\beta}^\top \mathbf{x}), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Funzione obiettivo (cross-entropy + regolarizzazione L2):

$$\mathcal{L}(\boldsymbol{\beta}) = - \sum_{i=1}^N \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right] + \frac{1}{2C} \|\boldsymbol{\beta}\|_2^2$$

dove $C = 1.0$ è il parametro di regolarizzazione inversa (maggiore C = minore regolarizzazione).

```

1 Pipeline([
2     ("scaler", StandardScaler()),
3     ("clf", LogisticRegression(
4         C=1.0,
5         solver="lbfgs",          # ottimizzatore quasi-Newton, efficiente per
                                   D << N
6         max_iter=1000,          # iterazioni massime convergenza
7         class_weight="balanced",
8         random_state=RANDOM_SEED, # 42
9     ))
10 ])
```

Listing 17: Configurazione Pipeline Logistic Regression

Il vettore $\boldsymbol{\beta} \in \mathbb{R}^{105}$ fornisce interpretabilità diretta: $\beta_j > 0$ indica che la feature j è positivamente associata alla classe vantaggiosa (C/D), $\beta_j < 0$ alla classe svantaggiosa (A/B). L'ordinamento per $|\beta_j|$ produce il ranking delle feature più discriminanti secondo il modello lineare. Il solver **lbfgs** (*Limited-memory Broyden-Fletcher-Goldfarb-Shanno*) è efficiente per problemi di piccole-medie dimensioni ($D = 105$) con regolarizzazione L2.

9.2 Support Vector Machine – kernel RBF

L'SVM con kernel RBF proietta implicitamente i dati in uno spazio di Hilbert a dimensione infinita tramite il *kernel trick*, trovando un iperpiano separatore di massimo margine in quello spazio:

SVM – Kernel RBF

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|_2^2\right)$$

con $\gamma = \frac{1}{D \cdot \text{Var}(X)}$ (**scale**), dove $D = 105$ e $\text{Var}(X)$ è la varianza media delle feature sul training set.

Problema di ottimizzazione primale:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i \quad \text{s.t.} \quad y_i (\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

```

1 Pipeline([
2     ("scaler", StandardScaler()),
3     ("clf", SVC(
4         C=1.0,
5         kernel="rbf",
6         gamma="scale",          # 1 / (n_features * X.var())
7     ))
8 ])
```

```

7         probability=True,          # calibrazione Platt per predict_proba e
           ROC-AUC
8         class_weight="balanced",
9         random_state=RANDOM_SEED,
10    ))
11 ])
```

Listing 18: Configurazione Pipeline SVM-RBF

Il parametro `probability=True` abilita la calibrazione di Platt (*Platt scaling*): addestra un modello logistico sigmoidale sulle *decision function* scores per produrre probabilità calibrate, necessarie per il calcolo della ROC-AUC tramite `predict_proba`.

Nota computazionale – SVM con kernel RBF

La complessità di training dell'SVM con kernel RBF è $O(N^2)$ – $O(N^3)$ rispetto al numero di campioni di training N . Con ≈ 9800 campioni di training per fold LOSO (54×200 trial), il training di un singolo fold richiede circa 10–30s su CPU consumer (Core i7 4-core). Con 55 fold e 3 modelli, il costo totale dell'SVM è dominante. La parallelizzazione LOSO (Sezione 10) mitiga significativamente questo costo.

9.3 Random Forest – ensemble bagging

Il Random Forest è un ensemble di $N_{\text{trees}} = 200$ alberi decisionali CART, addestrati su bootstrap samples dei dati e su sottoinsiemi casuali delle feature (*random subspace method*):

Random Forest – Predizione per voto di maggioranza

$$\hat{y}(\mathbf{x}) = \text{mode}\{T_1(\mathbf{x}), T_2(\mathbf{x}), \dots, T_{200}(\mathbf{x})\}$$

dove ogni albero T_k è addestrato su un bootstrap sample $\mathcal{D}_k \sim \mathcal{D}$ con replacement, usando $m = \lfloor \sqrt{D} \rfloor = \lfloor \sqrt{105} \rfloor = 10$ feature candidate per ogni split.

```

1 Pipeline([
2     ("scaler", StandardScaler()),
3     ("clf", RandomForestClassifier(
4         n_estimators=200,          # numero di alberi
5         max_depth=None,          # alberi non potati (regolarizzati da
           min_samples)
6         min_samples_split=5,     # minimo campioni per dividere un nodo
           interno
7         min_samples_leaf=2,      # minimo campioni in ogni foglia
8         class_weight="balanced",
9         n_jobs=-1,               # parallelismo interno RF su tutti i core
10        random_state=RANDOM_SEED,
11    ))
12 ])
```

Listing 19: Configurazione Pipeline Random Forest

Il Random Forest produce come sottoprodotto la stima dell'importanza delle feature tramite *mean decrease impurity* (Gini importance):

$$\text{Importance}(f_j) = \frac{1}{N_{\text{trees}}} \sum_{k=1}^{N_{\text{trees}}} \sum_{t \in T_k: \text{split on } f_j} \frac{n_t}{N} \left[\text{Gini}(t) - \frac{n_{t_L}}{n_t} \text{Gini}(t_L) - \frac{n_{t_R}}{n_t} \text{Gini}(t_R) \right]$$

dove n_t è il numero di campioni nel nodo t e $\text{Gini}(t) = 1 - \sum_k p_k^2$ è l'impurità Gini.

9.4 Pipeline sklearn e prevenzione del data leakage

Uno degli aspetti metodologicamente più critici è l'integrazione dello `StandardScaler` nella pipeline di cross-validazione. Lo scaler deve essere fittato *esclusivamente* sui dati di training di ogni fold:

```

1 # =====
2 # SBAGLIATO: scaler vede TUTTO il dataset (incluso il test)
3 # =====
4 scaler = StandardScaler()
5 scaler.fit(X_all) # media e std calcolate su training+test!
6 X_scaled = scaler.transform(X_all) # DATA LEAKAGE IMPLICITO
7 # Conseguenza: le statistiche di normalizzazione "vedono il futuro"
8 # --> performance ottimisticamente distorte nella CV
9
10 # =====
11 # CORRETTO: scaler DENTRO la Pipeline sklearn
12 # =====
13 pipeline = Pipeline([
14     ("scaler", StandardScaler()), # fittato solo su X_train
15     ("clf", RandomForestClassifier()),
16 ])
17 # Durante cross-validation:
18 # pipeline.fit(X_train, y_train) --> scaler.fit_transform(X_train)
19 # pipeline.predict(X_test) --> scaler.transform(X_test)
20 # con parametri del SOLO train set

```

Listing 20: Data leakage implicito vs approccio corretto con Pipeline sklearn

L'incapsulamento dello scaler nella `sklearn.Pipeline` garantisce automaticamente che le statistiche di normalizzazione (μ , σ per feature) vengano calcolate solo sui dati di training di ogni fold e applicate ai dati di test con i parametri del training set, senza alcuna forma di data leakage.

10 Validazione: Leave-One-Subject-Out (LOSO)

10.1 Motivazione della strategia LOSO

Perché la LOSO è necessaria per dati EEG multi-soggetto

I dati EEG sono fortemente soggetto-dipendenti: ogni individuo ha pattern spettrali unici legati a:

- morfologia individuale del cranio (diverso spessore osseo e conduttività);
- frequenza alpha individuale (IAF), che varia tra 8 Hz e 13 Hz tra soggetti;
- impedenza degli elettrodi (variabile tra sessioni);
- stili cognitivi individuali (attività theta basale diversa).

Una divisione random train/test che mescola trial dello stesso soggetto tra training e test set produce stime ottimisticamente distorte della performance: il classificatore impara le caratteristiche individuali del soggetto (fingerprint EEG), non le caratteristiche della decisione. Questo è un **data leakage inter-soggetto** che porta a performance irrealisticamente alte.

La **LOSO** garantisce che ogni fold utilizzi un soggetto distinto come test set. Con 55 soggetti, produce 55 fold indipendenti in cui il modello non ha mai visto il soggetto di test

durante il training.

10.2 Implementazione con LeaveOneGroupOut

La LOSO è implementata tramite `sklearn.model_selection.LeaveOneGroupOut` con `groups=subject_ids`, dove `subject_ids` è un array di stringhe con il codice soggetto per ogni riga di **X**:

```

1 from sklearn.model_selection import LeaveOneGroupOut
2
3 cv = LeaveOneGroupOut()
4 for fold_idx, (train_idx, test_idx) in enumerate(
5     cv.split(X, y, groups=subject_ids)):
6
7     # GARANZIA: nessun trial del soggetto test e' nel training set
8     X_train, X_test = X[train_idx], X[test_idx]
9     y_train, y_test = y[train_idx], y[test_idx]
10
11     subj_test = subject_ids[test_idx[0]] # es. "s-07"
12     logger.info(f"  Fold {fold_idx+1}/55: test su {subj_test} "
13               f"({len(test_idx)} trial)")
14
15     # Pipeline sklearn: StandardScaler fittato solo su X_train
16     pipeline_clone = clone(pipeline) # copia fresca
17     pipeline_clone.fit(X_train, y_train)
18     y_pred = pipeline_clone.predict(X_test)
19     y_proba = pipeline_clone.predict_proba(X_test)[: , 1]
```

Listing 21: LOSO cross-validation con scikit-learn LeaveOneGroupOut

10.3 Parallelizzazione dei fold – `run_cross_validation()`

Con $55 \text{ fold} \times 3 \text{ modelli}$, l'esecuzione sequenziale richiederebbe:

$$T_{\text{seq}} \approx 55 \times 3 \times \bar{t}_{\text{fold}} \approx 165 \times 20 \text{ s} \approx 55 \text{ min}$$

su hardware consumer (stima per SVM su ≈ 9800 campioni). I fold LOSO sono matematicamente indipendenti (ogni fold è un esperimento separato), rendendo la parallelizzazione triviale e senza impatto sui risultati. La funzione `run_cross_validation()` usa un meccanismo di fallback per garantire compatibilità:

```

1 # Fallback: sklearn.utils.parallel (>= 1.3) oppure joblib
2 try:
3     from sklearn.utils.parallel import Parallel, delayed
4 except ImportError:
5     from joblib import Parallel, delayed
6
7 fold_results_list = Parallel(n_jobs=n_jobs, verbose=0)(
8     delayed(_run_single_fold)(
9         fold_idx, train_idx, test_idx,
10        X, y, subject_ids,
11        models, # dict {nome: Pipeline}
12        total_folds
13    )
14    for fold_idx, (train_idx, test_idx) in enumerate(all_splits)
15 )
```

Listing 22: Parallelizzazione LOSO con fallback sklearn/joblib

Funzione helper `_run_single_fold()`

Ogni chiamata parallela invoca `_run_single_fold()` che:

1. Clona ogni pipeline tramite `sklearn.base.clone()` per garantire indipendenza tra processi;
2. Verifica che il test set contenga almeno due classi (altrimenti skipa il fold con warning);
3. Addestra ogni pipeline clonata su `(X_train, y_train)`;
4. Calcola le 5 metriche su `(X_test, y_test)`;
5. Collezione i dati ROC (`fpr`, `tpr`, `auc`) per il plot ROC medio;
6. Accumula la confusion matrix per il plot CM aggregata;
7. Restituisce un dizionario con tutti i risultati del fold.

L'uso di `clone()` garantisce:

- **Assenza di race condition:** ogni processo worker ha il proprio stato isolato;
- **Correttezza statistica:** ogni fold parte da un modello non addestrato;
- **Riproducibilità:** `random_state=42` nei modelli garantisce lo stesso risultato in ogni run.

Backend `loky` e Global Interpreter Lock (GIL)

Il backend `loky` (process-based) di `joblib` è scelto in modo predefinito invece di `threading`. Il motivo è il Global Interpreter Lock (GIL) di CPython: il GIL impedisce l'esecuzione simultanea di bytecode Python in thread diversi dello stesso processo, limitando il parallelismo a thread per codice CPU-bound (come il training ML). Il backend `loky` usa processi separati (non thread) e non è soggetto al GIL, garantendo vero parallelismo su tutti i core disponibili. È il backend raccomandato per portabilità su Windows (dove il backend `fork` non è disponibile), macOS e Linux.

10.4 GroupKFold come alternativa

In alternativa alla LOSO, è disponibile la `GroupKFold` con k configurabile tramite `-n_splits`. La funzione seleziona automaticamente la strategia:

```

1 if cv_strategy == "loso":
2     cv = LeaveOneGroupOut()
3     n_folds = len(np.unique(subject_ids)) # = 55
4     logger.info(f"Strategia: LOSO ({n_folds} fold)")
5 elif cv_strategy == "group_kfold":
6     cv = GroupKFold(n_splits=n_splits) # default k=5
7     n_folds = n_splits
8     logger.info(f"Strategia: GroupKFold (k={n_splits})")

```

Listing 23: Selezione automatica della strategia CV in `run_cross_validation()`

Con $k = 5$, ogni fold include ≈ 11 soggetti nel test set. È computazionalmente $\approx 11\times$ più veloce della LOSO ma produce stime meno accurate della generalizzazione cross-subject. È raccomandata per:

- verificare rapidamente il corretto funzionamento della pipeline;
- debugging con `-limit N` su pochi soggetti;
- prototipazione di nuove feature prima della LOSO completa.

11 Metriche di Valutazione

11.1 Metriche calcolate per fold

Per ogni fold e per ogni modello vengono calcolate cinque metriche di classificazione. La classe positiva è la classe 1 (decisione vantaggiosa C/D):

Table 15: Definizione e interpretazione delle cinque metriche di valutazione

Metrica	Formula	Interpretazione nel contesto IGT
Accuracy	$(TP + TN)/(TP + TN + FP + FN)$	Proporzione complessiva di trial correttamente classificati (advantageous e disadvantageous)
Precision	$TP/(TP + FP)$	Tra i trial predetti come vantaggiosi (C/D), quanti erano effettivamente vantaggiosi
Recall	$TP/(TP + FN)$	Tra i trial effettivamente vantaggiosi (C/D), quanti vengono correttamente identificati
F1-Score	$\frac{2 \cdot P \cdot R}{P + R}$	Media armonica Precision e Recall; bilancia le due metriche
ROC-AUC	$\int_0^1 \text{TPR} d(\text{FPR})$	Area sotto la curva ROC; metrica principale robusta allo sbilanciamento

Le metriche vengono calcolate nel corpo di `_run_single_fold()` con le funzioni di `sklearn.metrics`:

```

1 from sklearn.metrics import (
2     accuracy_score, precision_score, recall_score,
3     f1_score, roc_auc_score, roc_curve
4 )
5
6 metrics = {
7     "accuracy": accuracy_score(y_test, y_pred),
8     "precision": precision_score(y_test, y_pred, zero_division=0),
9     "recall": recall_score(y_test, y_pred, zero_division=0),
10    "f1": f1_score(y_test, y_pred, zero_division=0),
11    "roc_auc": roc_auc_score(y_test, y_proba) # usa probabilita'
12                if len(np.unique(y_test)) > 1 else np.nan,
13 }
14
15 # Dati ROC per il plot della curva media
16 fpr, tpr, _ = roc_curve(y_test, y_proba)
17 roc_entry = (fpr, tpr, metrics["roc_auc"])

```

Listing 24: Calcolo delle cinque metriche per fold in `_run_single_fold()`

Il parametro `zero_division=0` evita warning per fold in cui il classificatore predice sempre la stessa classe (P=0/0 o R=0/0). Il calcolo della ROC-AUC viene saltato (restituisce `np.nan`) se il test set contiene una sola classe.

Ruolo della ROC-AUC come metrica principale

La ROC-AUC è scelta come metrica principale per i seguenti motivi:

1. **Robustezza allo sbilanciamento:** misura il potere discriminante indipendentemente dalla proporzione di classi nel test set, che varia tra soggetti (da 33/167 a 167/33);
2. **Indipendenza dalla soglia:** valuta la performance su tutte le possibili soglie di decisione, non solo quella di default (0.5);
3. **Interpretazione probabilistica:** $AUC = P(\hat{p}_{C/D}^{(i)} > \hat{p}_{C/D}^{(j)})$ per un trial vantaggioso i e svantaggioso j campionati casualmente;
4. **Valore di riferimento:** $AUC = 0.5$ corrisponde esattamente al classificatore casuale; $AUC = 1.0$ alla classificazione perfetta.

11.2 Aggregazione e tabella riepilogativa – `compute_summary_table()`

Aggregazione delle metriche sui 55 fold LOSO

$$\bar{\mu}_{\text{metric}} = \frac{1}{N_{\text{val}}} \sum_{i \in \mathcal{V}} m_i, \quad \sigma_{\text{metric}} = \sqrt{\frac{1}{N_{\text{val}}} \sum_{i \in \mathcal{V}} (m_i - \bar{\mu})^2}$$

dove $\mathcal{V} \subseteq \{1, \dots, 55\}$ è il sottoinsieme dei fold validi (esclude fold con NaN, cioè fold con una sola classe nel test set).

I risultati vengono salvati in `results_summary.csv` con colonne `{metrica}_mean` e `{metrica}_std` per ogni modello (righe). La tabella viene stampata a terminale tramite `rich.Table` con colorazione semantica delle celle (applicata a media e deviazione standard):

- **Verde grassetto:** valori ≥ 0.75 (performance buona);
- **Giallo:** valori nell'intervallo $[0.60, 0.75)$ (performance accettabile);
- **Rosso:** valori < 0.60 (performance vicina al caso – attesa per questo task).

12 Visualizzazioni dei Risultati ML

Tutte le visualizzazioni ML usano una configurazione di stile *light theme* per qualità di pubblicazione scientifica, impostata dalla funzione `_setup_plot_style()`:

```

1 def _setup_plot_style():
2     plt.rcParams.update({
3         "figure.facecolor": "white",
4         "axes.facecolor": "#F7F9FC",    # sfondo assi grigio chiaro
5         "axes.grid": True,
6         "grid.alpha": 0.4,
7         "grid.linestyle": "--",
8         "axes.spines.top": False,        # spine superiore nascosta
9         "axes.spines.right": False,      # spine destra nascosta
10        "font.family": "sans-serif",
11        "font.size": 10,
12    })

```

Listing 25: Configurazione stile globale matplotlib per grafici ML

12.1 Confusion Matrix aggregata – `plot_confusion_matrices()`

La CM aggregata viene costruita accumulando le predizioni di tutti i fold LOSO su tutti gli 11.000 trial (non la media delle CM per fold), poi normalizzata per riga (recall per classe reale):

$$\widetilde{CM}_{ij} = \frac{CM_{ij}}{\sum_{j'} CM_{ij'}} \quad (\text{normalizzazione per classe reale} = \text{recall})$$

La figura presenta 1×3 subplot (uno per modello) generati tramite `seaborn.heatmap` con:

- colormap Blues proporzionale alla proporzione normalizzata;
- annotazioni doppie in ogni cella: percentuale normalizzata (16pt bold) e conteggio assoluto in grigio (9pt);
- colore del testo adattivo (bianco se sfondo > 0.58 , nero altrimenti) per leggibilità;
- accuracy globale per ogni modello riportata sotto il rispettivo subplot.

12.2 ROC Curve media con deviazione standard – `plot_roc_curves()`

La curva ROC media viene costruita interpolando le TPR di ogni fold su una griglia comune di 300 punti equidistanti in $[0, 1]$ per FPR:

```

1 mean_fpr = np.linspace(0, 1, 300)
2 tprs_interp = []
3
4 for (fpr, tpr, auc_val) in fold_rocs[model_name]:
5     interp = np.interp(mean_fpr, fpr, tpr)
6     interp[0] = 0.0 # forza partenza da (0, 0)
7     tprs_interp.append(interp)
8
9 mean_tpr = np.mean(tprs_interp, axis=0)
10 mean_tpr[-1] = 1.0 # forza arrivo a (1, 1)
11 std_tpr = np.std(tprs_interp, axis=0)
12 mean_auc = np.mean([r[2] for r in fold_rocs[model_name]])
13
14 # Curva media
15 ax.plot(mean_fpr, mean_tpr, color=color, lw=2.0,
16         label=f"{model_name}: AUC={mean_auc:.3f}")
17 # Banda +/- 1 std
18 ax.fill_between(mean_fpr,
19                 np.clip(mean_tpr - std_tpr, 0, 1),
20                 np.clip(mean_tpr + std_tpr, 0, 1),
21                 color=color, alpha=0.15)
```

Listing 26: Costruzione ROC curve media su griglia comune – `plot_roc_curves()`

I tre modelli sono sovrapposti in un unico grafico con la palette `PALETTE = ["#1A56A0", "#C0392B", "#1E8449"]` (blu, rosso, verde). La linea di riferimento casuale ($AUC = 0.5$) è mostrata come linea grigia tratteggiata. Un marcatore circolare è posizionato sul punto mediano ($FPR = 0.3$) di ogni curva per facilitare il confronto visivo.

12.3 Feature Importance – `plot_feature_importance()`

Visualizza le top- N feature per importanza (default $N = 20$) con due subplot affiancati:

- **Random Forest (subplot sinistro):** importanza Gini media sui 200 alberi con barre di errore = deviazione standard tra gli alberi (accessibile tramite `clf.feature_importances_` e calcolata dalla variance degli alberi individuali). Il colore delle barre è proporzionale all'importanza (colormap Blues).
- **Logistic Regression (subplot destro):** coefficienti $|\beta_j|$ ordinati in senso decrescente. Le barre sono colorate in blu se $\beta_j > 0$ (feature associata alla classe C/D, vantaggiosa) e

in rosso se $\beta_j < 0$ (associata a A/B, svantaggiosa). L'opacità è proporzionale all'entità del coefficiente.

Nota metodologica critica – Feature Importance $\neq$ Feature Selection

Il plot di feature importance è prodotto a **scopo puramente descrittivo**: i modelli RF e LR vengono riadattati sull'intero dataset (**X** completo) dopo la cross-validation LOSO, esclusivamente per estrarre l'importanza delle feature a scopo interpretativo. Questo calcolo avviene *dopo* la CV, non prima.

Attenzione: usare l'importanza delle feature per selezionare un sottoinsieme di feature *prima* della cross-validation causerebbe data leakage: la selezione sarebbe influenzata dall'intero dataset (inclusi i test set di ogni fold), producendo stime ottimisticamente distorte.

Il file `feature_ranking.csv` viene salvato con colonne: `feature_name`, `rf_importance`, `rf_importance_std`, `lr_coef` (per tutte le 105 feature, non solo le top-20).

12.4 Distribuzione metriche per fold – `plot_per_fold_metrics()`

Genera 5 subplot (uno per metrica: Accuracy, Precision, Recall, F1, ROC-AUC) con la distribuzione delle metriche per fold e per modello. La modalità di visualizzazione si adatta automaticamente al numero di fold:

```

1 # Violin con KDE solo se ci sono abbastanza fold per stimare la distribuzione
2 use_violin = (n_valid_folds >= 8)
3
4 if use_violin:
5     ax.violinplot([data[m] for m in model_names],
6                   positions=positions, showmedians=False)
7     # Mediana manuale come linea nera spessa
8     for pos, mn in zip(positions, model_names):
9         med = np.nanmedian(data[mn])
10        ax.hlines(med, pos - 0.15, pos + 0.15, colors="black", lw=2)
11 else:
12     ax.boxplot([data[m] for m in model_names], positions=positions)

```

Listing 27: Selezione automatica violin vs box plot in `plot_per_fold_metrics()`

In entrambi i casi, un `seaborn.stripplot()` sovrapposto rende visibili i singoli fold come punti (jitter = 0.15 per evitare sovrapposizione). Una linea orizzontale tratteggiata a $y = 0.5$ (livello chance) è inclusa in ogni subplot. Sotto ogni gruppo viene annotato il numero di fold validi e la media.

13 Generazione Dati Sintetici – `generate_synthetic_data()`

13.1 Motivazione e struttura

La funzione `generate_synthetic_data()` in `eeg_ml_pipeline.py` genera dati EEG simulati realistici che permettono di testare la pipeline completa senza scaricare il dataset reale di Mendeley. I dati sintetici sono progettati per contenere una differenza di classe simulata che il classificatore può rilevare, consentendo la verifica del corretto funzionamento dell'intera catena.

13.2 Struttura dei dati sintetici

Per ogni soggetto sintetico, la funzione genera:

- **200 epoche** di shape (21, 513) – rumore gaussiano $\mathcal{N}(0, 1)$ con sovrapposizione di segnali sinusoidali;
- **Differenza di classe simulata:**
 - Classe 0 (svantaggioso): potenziamento theta frontale (6 Hz) su Fz, F3, F4 – $\Delta \approx +0.5$ SNR;
 - Classe 1 (vantaggioso): potenziamento alpha occipitale (10 Hz) su O1, O2 – $\Delta \approx +0.5$ SNR.
- **Rumore aggiuntivo per soggetto:** variabilità inter-soggetto simulata tramite scaling casuale dell'ampiezza ($\sigma_{\text{subj}} \sim \mathcal{U}(0.8, 1.2)$);
- **File CSV compatibili:** output nel formato identico al dataset reale (s-XX_eeg.csv, s-XX_igt.csv) per consentire l'uso diretto di `run_full_pipeline()`.

L'AUC attesa sui dati sintetici è nell'intervallo 0.55–0.65 (leggermente superiore al dataset reale), confermando che la pipeline rileva la differenza simulata.

14 Caricamento e Salvataggio Modelli – `load_preprocessed_data()` e `save_models()`

14.1 Caricamento – `load_preprocessed_data()`

```

1 def load_preprocessed_data(input_dir):
2     """
3     Carica tutti i file *_epochs.npy dalla directory di preprocessing.
4
5     Returns
6     -----
7     epochs_data : list of np.ndarray, ciascuno shape (n_ep, 21, 513)
8     labels       : np.ndarray (N_tot,), etichette binarie
9     subject_ids  : np.ndarray (N_tot,), ID soggetto per ogni trial
10    """
11    epochs_dir = Path(input_dir) / "epochs"
12    if not epochs_dir.is_dir():
13        epochs_dir = Path(input_dir) # fallback: cerca direttamente
14
15    epoch_files = sorted(epochs_dir.glob("*_epochs.npy"))
16    if not epoch_files:
17        raise FileNotFoundError(f"Nessun file *_epochs.npy in {epochs_dir}")
18
19    epochs_data, all_labels, all_subj = [], [], []
20    for ep_file in epoch_files:
21        sid = ep_file.stem.replace("_epochs", "") # es. "s-01"
22        label_file = ep_file.parent / f"{sid}_labels.npy"
23        epochs = np.load(ep_file) # (n_ep, 21, 513)
24        labels = np.load(label_file) # (n_ep,)
25        epochs_data.append(epochs)
26        all_labels.extend(labels.tolist())
27        all_subj.extend([sid] * len(labels))
28        logger.info(f"Caricato {sid}: {epochs.shape}, labels={labels.shape}")
29
30    labels = np.array(all_labels, dtype=int)
31    subject_ids = np.array(all_subj)
32    return epochs_data, labels, subject_ids

```

Listing 28: Implementazione `load_preprocessed_data()` in `eeg_ml_pipeline.py`

14.2 Salvataggio modelli finali – save_models()

```

1 def save_models(X, y, models, output_dir):
2     """
3     Riaddestra ogni modello sull'INTERO dataset e salva come .joblib.
4     Usare per deployment (non per valutazione -- quella e' la LOSO CV).
5     """
6     models_dir = Path(output_dir) / "saved_models"
7     models_dir.mkdir(parents=True, exist_ok=True)
8
9     for name, pipeline in models.items():
10        pipeline_final = clone(pipeline)
11        pipeline_final.fit(X, y) # training su TUTTI i dati
12        safe_name = name.lower().replace(" ", "_").replace("-", "_")
13        save_path = models_dir / f"{safe_name}.joblib"
14        joblib.dump(pipeline_final, save_path)
15        logger.info(f" Modello salvato: {save_path.name}")

```

Listing 29: Implementazione save_models() per deployment

Nota – Modelli finali vs modelli per valutazione

I modelli salvati in `saved_models/*.joblib` sono addestrati sull'intero dataset (11.000 trial, tutti i 55 soggetti) e sono destinati al **deployment** (uso su nuovi soggetti non visti). **Non devono essere usati per la valutazione della performance**: quella è fornita dalla LOSO cross-validation sui 55 fold, che garantisce la separazione rigorosa tra training e test. Le performance riportate in `results_summary.csv` sono quelle della LOSO, non dei modelli finali.

15 Orchestratore Unificato (run_pipeline.py)

15.1 Panoramica e motivazione architetturale

Il modulo `run_pipeline.py` coordina in sequenza tutti gli step della pipeline end-to-end, dalla generazione o caricamento dei dati fino alla produzione dei risultati finali ML, in un singolo comando da terminale. La filosofia progettuale del modulo è quella del *thin orchestrator*: non implementa logica di analisi propria, ma si limita a coordinare l'esecuzione dei moduli specializzati.

I tre step principali sono implementati come funzioni autonome (`step_generate_synthetic()`, `step_preprocessing()`, `step_ml()`) che importano direttamente i moduli come librerie Python – non come sottoprocessi tramite `subprocess.run()` o simili. Questa scelta garantisce:

- **Nessun overhead di serializzazione inter-processo**: i dati numerici (array NumPy) non vengono serializzati e deserializzati tra processi;
- **Logging unificato**: tutti i messaggi di log dei tre moduli vengono incanalati nello stesso handler `RichHandler`, producendo un output terminale coerente con timestamp e livello di log;
- **Gestione centralizzata degli errori**: le eccezioni non catturate nei moduli propagano all'orchestratore che le gestisce con messaggi diagnostici contestualizzati;
- **Possibilità di test indipendente**: ogni step può essere chiamato direttamente da notebook Jupyter importando `run_pipeline`.

15.2 Configurazione del logging unificato

```

1 import logging
2 from rich.logging import RichHandler
3
4 logging.basicConfig(
5     level=logging.INFO,
6     format="%(message)s",
7     datefmt="[%H:%M:%S]",
8     handlers=[RichHandler(
9         rich_tracebacks=True,      # traceback formattati con Rich
10        markup=True,                # supporto markup Rich nei messaggi
11        show_path=False             # nasconde il percorso del file sorgente
12    )]
13 )
14 logger = logging.getLogger("orchestrator")
15 console = Console()                # istanza Rich Console per pannelli e tabelle

```

Listing 30: Configurazione logging unificato con RichHandler

15.3 Funzione _section(title)

Funzione helper che stampa un pannello Rich visivamente distinto per separare le sezioni principali dell'output terminale:

```

1 def _section(title: str) -> None:
2     """Stampa un pannello Rich colorato per separare le sezioni."""
3     console.print()
4     console.print(Panel(
5         f"[bold white]{title}[/bold white]",
6         border_style="#1A56A0",    # blu Insubria
7         expand=False,
8         padding=(0, 2),
9     ))
10    console.print()

```

Listing 31: Helper _section() per separatori visivi nel terminale

15.4 Step 0 – Generazione dataset sintetico

`step_generate_synthetic(output, n_subjects, seed)`

Disponibile esclusivamente in modalità `-mode synthetic`. Invoca la funzione `generate_dataset()` dal modulo `src.generate_synthetic_dataset` e restituisce il percorso assoluto della directory creata. La funzione stampa il pannello di sezione con il numero di soggetti da generare, misura il tempo di esecuzione e logga il tempo trascorso al completamento.

```

1 def step_generate_synthetic(output: str, n_subjects: int, seed: int) -> str:
2     from src.generate_synthetic_dataset import generate_dataset
3
4     _section(f"STEP 0 - Generazione dataset sintetico ({n_subjects}
5             soggetti)")
6     t0 = time.time()
7
8     generate_dataset(
9         output_root=output,
10        n_subjects=n_subjects,
11        seed=seed,
12    )

```

```

13     elapsed = time.time() - t0
14     logger.info(f"Dataset sintetico generato in {elapsed:.1f}s ({n_subjects}
        soggetti)")
15     return output      # percorso directory creata

```

Listing 32: Implementazione step_generate_synthetic()

15.5 Step 1 – Preprocessing

step_preprocessing(dataset_root, output_dir, save_figures, subject_limit)

Invoca run_full_pipeline() da eeg_preprocessing.py in-process. Se nessun soggetto viene elaborato correttamente (es. dataset vuoto o tutti i soggetti in errore), lancia RuntimeError con messaggio diagnostico che suggerisce di verificare i file di input.

```

1  def step_preprocessing(
2      dataset_root: str, output_dir: str,
3      save_figures: bool, subject_limit: Optional[int]
4  ) -> str:
5      from src.eeg_preprocessing import run_full_pipeline
6
7      _section("STEP 1 - Preprocessing EEG")
8      t0 = time.time()
9
10     summary_df = run_full_pipeline(
11         dataset_root=dataset_root,
12         output_dir=output_dir,
13         save_figures=save_figures,
14         subject_limit=subject_limit,
15     )
16
17     n_ok      = len(summary_df)
18     elapsed = time.time() - t0
19     logger.info(f"Preprocessing completato: {n_ok} soggetti in
        {elapsed:.1f}s")
20
21     if n_ok == 0:
22         raise RuntimeError(
23             "Nessun soggetto elaborato correttamente. "
24             "Controlla i file di input nella directory del dataset."
25         )
26     return output_dir      # percorso directory epoche

```

Listing 33: Implementazione step_preprocessing()

15.6 Step 2 – Machine Learning

step_ml(input_dir, output_dir, cv_strategy, n_splits, n_jobs, save_models)

Invoca load_preprocessed_data() e run_pipeline() da eeg_ml_pipeline.py. Prima del caricamento scansiona la directory per riportare nel log il numero di file *_epochs.npy trovati. Al termine, stampa la tabella riepilogativa dei risultati tramite rich.Table con colorazione semantica.

```

1  def step_ml(
2      input_dir: str, output_dir: str,
3      cv_strategy: str, n_splits: int,
4      n_jobs: int, save_models: bool
5  ) -> None:
6      from src.eeg_ml_pipeline import load_preprocessed_data, run_pipeline

```

```

7
8     _section("STEP 2 - Feature Extraction + Machine Learning")
9     t0 = time.time()
10
11     # Conta i file epoch prima del caricamento (feedback utente)
12     _ep_dir = Path(input_dir) / "epochs"
13     _ep_search = _ep_dir if _ep_dir.is_dir() else Path(input_dir)
14     _n_files = len(list(_ep_search.glob("*_epochs.npy")))
15     logger.info(f"Caricamento epoche preprocessate ({_n_files} soggetti)...")
16
17     epochs_data, labels, subject_ids = load_preprocessed_data(input_dir)
18
19     summary_df = run_pipeline(
20         epochs_data=epochs_data, labels=labels, subject_ids=subject_ids,
21         output_dir=output_dir, cv_strategy=cv_strategy,
22         n_splits=n_splits, n_jobs=n_jobs,
23         save_trained_models=save_models,
24     )
25
26     elapsed = time.time() - t0
27     logger.info(f"ML pipeline completata in {elapsed:.1f}s")
28
29     # Tabella Rich con colorazione semantica delle metriche
30     metric_keys = ["accuracy", "precision", "recall", "f1", "roc_auc"]
31     table = Table(
32         title=f"[bold white]Risultati Finali - {cv_strategy.upper()}[/bold white]",
33         box=rbox.HEAVY_HEAD, show_header=True,
34         header_style="bold white on #1A56A0",
35         border_style="#5B8DD9", expand=False, padding=(0, 1),
36     )
37     table.add_column("Modello", style="bold", min_width=22, no_wrap=True)
38     for ml in ["Accuracy", "Precision", "Recall", "F1-Score", "ROC-AUC"]:
39         table.add_column(ml, justify="center", min_width=15)
40
41     def _style_cell(v: float) -> str:
42         if np.isnan(v): return "dim"
43         if v >= 0.75: return "bold green"
44         if v >= 0.60: return "yellow"
45         return "red"
46
47     for model in summary_df.index:
48         cells = [model]
49         for m in metric_keys:
50             mean_v = summary_df.loc[model, f"{m}_mean"]
51             std_v = summary_df.loc[model, f"{m}_std"]
52             cells.append(Text(f"{mean_v:.3f} +/- {std_v:.3f}",
53                             style=_style_cell(mean_v)))
54         table.add_row(*cells)
55
56     console.print(); console.print(table); console.print()

```

Listing 34: Implementazione step_ml() con tabella Rich dei risultati finali

15.7 Funzione main() – Parsing argomenti e flusso principale

La funzione main() è il punto di ingresso del modulo quando eseguito come script. Implementa i seguenti step:

1. **Parsing argomenti:** tramite `argparse.ArgumentParser` con gruppi separati per `-mode`, `input`, `output`, dati sintetici, preprocessing e ML;

2. **Validazione:** verifica che `-dataset` sia specificato se `-mode real`; lancia `parser.error()` con messaggio diagnostico in caso contrario;
3. **Cambio directory di lavoro:** esegue `os.chdir(script_dir)` e aggiunge `script_dir` a `sys.path` per garantire che gli import relativi (`src.*`) funzionino correttamente indipendentemente dalla directory da cui viene invocato lo script;
4. **Pannello di benvenuto:** stampa un pannello Rich con modalità, percorso output e timestamp;
5. **Esecuzione step:** chiama in sequenza i tre step con timing per ogni fase;
6. **Pannello finale:** stampa il tempo totale di esecuzione e il percorso dell'output.

```

1 # Pannello di benvenuto
2 console.print(Panel(
3     f"[bold white]ORCHESTRATORE EEG-IGT[/bold white]\n"
4     f"Modalita' : [cyan]{args.mode.upper()}[/cyan]\n"
5     f"Output      : [dim]{root_out}[/dim]",
6     title="[bold #1A56A0]EEG-IGT Pipeline[/bold #1A56A0]",
7     border_style="#1A56A0", expand=False, padding=(0, 2),
8 ))
9
10 # ...esecuzione step ...
11
12 # Pannello di completamento
13 total_elapsed = time.time() - total_start
14 console.print(Panel(
15     f"[bold green]PIPELINE COMPLETA[/bold green] in "
16     f"[cyan]{total_elapsed:.1f}s[/cyan]\n"
17     f"Output : [dim]{root_out}[/dim]",
18     border_style="green", expand=False, padding=(0, 2),
19 ))

```

Listing 35: Pannello di benvenuto e pannello finale in `main()`

15.8 Struttura delle directory di output

```

1 pipeline_output/                                (--output-root, default:
   ./pipeline_output)
2 |
3 +-- data_synthetic/                             (solo con --mode synthetic)
4 |   +-- s-00/
5 |       |   +-- s-00_eeg.csv                     (EEG sintetico: n_samples x 21, uV)
6 |       |   +-- s-00_igt.csv                     (IGT sintetico: 200 trial)
7 |       +-- s-01/ ... s-09/                     (default: 10 soggetti)
8 |
9 +-- preprocessing/                             (output di eeg_preprocessing.py)
10 |   +-- epochs/
11 |       |   +-- s-XX_epochs.npy                 (200, 21, 513) float64
12 |       |   +-- s-XX_labels.npy                 (200,) int
13 |       |   +-- s-XX_samples.npy               (200,) int
14 |       |   +-- s-XX_info.npz                   metadati
15 |       +-- figures/                           (omessa con --no-figures)
16 |           +-- s-XX_raw_vs_clean.png
17 |           +-- s-XX_psd_pre_filter.png
18 |           +-- s-XX_psd_post_filter.png
19 |           +-- s-XX_erp_mean.png
20 |       +-- pipeline_summary.csv
21 |
22 +-- ml_results/                                (output di eeg_ml_pipeline.py)

```

```

23  +-- results_summary.csv          media +/- std per metrica e modello
24  +-- feature_matrix.npy          (N_tot, 105) float32
25  +-- feature_names.csv          105 nomi: Fz_theta, O1_alpha, ...
26  +-- feature_ranking.csv         importanza RF + coefficienti LR
27  +-- confusion_matrices.png      CM aggregata LOS0 (1x3 subplot)
28  +-- roc_curves.png              ROC media +/- std, 3 modelli
29  +-- feature_importance.png      top-20 feature: RF + LR
30  +-- metrics_boxplot.png         violin/box plot per fold
31  +-- saved_models/              (omessa con --no-save-models)
32      +-- logistic_regression.joblib
33      +-- svm_rbf.joblib
34      +-- random_forest.joblib

```

Listing 36: Struttura completa delle directory generate dall'orchestratore

16 Comandi di Esecuzione della Pipeline

16.1 Prerequisiti e posizionamento

La pipeline si avvia da terminale posizionandosi nella directory radice del progetto (`eeg_igt_pipeline/`). Gli import relativi (`from src.eeg_preprocessing import ...`) richiedono che la directory di lavoro corrente sia la radice del progetto. L'orchestratore esegue automaticamente `os.chdir(script_dir)` all'avvio per garantire questo requisito.

Differenze sintattiche tra sistemi operativi

Windows CMD: percorsi con backslash (`\`), carattere `^` per continuazione di riga, comando `python` (non `python3`).

macOS / Linux Terminal: percorsi con slash (`/`), carattere `\` per continuazione di riga.

Gli argomenti (`-mode`, `-dataset`, ecc.) sono identici su tutti i sistemi operativi.

Nota macOS: su sistemi con Python 2 ancora nel PATH, usare sempre `python3` invece di `python`. Verificare con `python -version`. Se necessario, attivare il virtual environment: `source venv/bin/activate`.

16.2 Orchestratore unificato – uso raccomandato

Test rapido con dati sintetici (nessun dataset richiesto)

```

1  # Genera 10 soggetti sintetici, preprocessa e classifica (default)
2  python run_pipeline.py --mode synthetic

```

Dati sintetici – configurazione personalizzata

```

1  # 20 soggetti, seed diverso, senza figure diagnostiche
2  python run_pipeline.py \
3      --mode synthetic \
4      --n_subjects 20 \
5      --seed 123 \
6      --no-figures

```

Dati sintetici – GroupKFold per verifica rapida

```

1  python run_pipeline.py \
2      --mode synthetic \

```

```
3 --n_subjects 15 \  
4 --cv group_kfold \  
5 --n_splits 3
```

Dataset reale Mendeley – LOSO completa

```
1 # Tutti i core disponibili, LOSO completa su 55 soggetti  
2 python run_pipeline.py \  
3 --mode real \  
4 --dataset ./data/igt_eeg_dataset
```

Dataset reale – LOSO limitando i core

```
1 # Usa 4 core (evita di bloccare il PC durante elaborazione)  
2 python run_pipeline.py \  
3 --mode real \  
4 --dataset ./data/igt_eeg_dataset \  
5 --n_jobs 4
```

Dataset reale – senza figure e senza salvataggio modelli

```
1 python run_pipeline.py \  
2 --mode real \  
3 --dataset ./data/igt_eeg_dataset \  
4 --no-figures \  
5 --no-save-models
```

Dataset reale – debug su 5 soggetti con GroupKFold

```
1 python run_pipeline.py \  
2 --mode real \  
3 --dataset ./data/igt_eeg_dataset \  
4 --limit 5 \  
5 --cv group_kfold \  
6 --n_splits 5
```

Output personalizzato

```
1 python run_pipeline.py \  
2 --mode real \  
3 --dataset ./data/igt_eeg_dataset \  
4 --output-root ./risultati_esperimento_1
```

16.3 Riepilogo argomenti dell'orchestratore

Table 16: Argomenti completi accettati da `run_pipeline.py`

Argomento	Tipo	Descrizione	Default
<code>-mode</code>	str	synthetic : genera dati e processa. real : usa dataset reale	synthetic
<code>-dataset</code>	str	Percorso dir. radice dataset reale (obbligatorio con real)	–
<code>-output-root</code>	str	Dir. radice per tutti gli output	<code>./pipeline_output</code>
<code>-n_subjects</code>	int	Soggetti sintetici da generare (solo synthetic)	10
<code>-seed</code>	int	Seed random per generazione sintetica	42
<code>-no-figures</code>	flag	Disabilita figure diagnostiche per soggetto	disattivo
<code>-limit</code>	int	Elabora solo i primi <i>N</i> soggetti (debug)	None
<code>-cv</code>	str	Strategia CV: loso o group_kfold	loso
<code>-n_splits</code>	int	<i>k</i> per GroupKFold (ignorato con loso)	5
<code>-n_jobs</code>	int	Processi paralleli (<code>-1</code> = tutti i core)	<code>-1</code>
<code>-no-save-models</code>	flag	Non salvare i modelli <code>.joblib</code> finali	disattivo

16.4 Uso diretto dei moduli (avanzato)

I moduli possono essere eseguiti autonomamente per scenari avanzati (es. preprocessing già completato, si vuole solo ripetere la fase ML):

Preprocessing standalone

```

1 python src/eeg_preprocessing.py \
2   --dataset ./data/igt_eeg_dataset \
3   --output  ./output_preproc \
4   --limit   5                # elabora solo i primi 5 soggetti

```

Listing 37: Preprocessing standalone su dataset reale

```

1 python src/eeg_preprocessing.py \
2   --dataset ./data/igt_eeg_dataset \
3   --output  ./output_preproc \
4   --no-figures

```

Listing 38: Preprocessing standalone senza figure

ML standalone – diverse strategie di validazione

```

1 python src/eeg_ml_pipeline.py \
2   --mode    real \
3   --input   ./output_preproc \
4   --output  ./ml_results_loso \
5   --cv      loso

```

Listing 39: LOSO completa su tutti i core

```

1 python src/eeg_ml_pipeline.py \
2   --mode    real \
3   --input   ./output_preproc \

```

```

4  --output ./ml_results_loso \
5  --cv     loso \
6  --n_jobs 4

```

Listing 40: LOSO con numero core limitato

```

1 python src/eeg_ml_pipeline.py \
2   --mode      real \
3   --input     ./output_preproc \
4   --output    ./ml_results_loso \
5   --cv        loso \
6   --no_save_models

```

Listing 41: LOSO senza salvare i modelli finali

```

1 python src/eeg_ml_pipeline.py \
2   --mode      real \
3   --input     ./output_preproc \
4   --output    ./ml_results_kfold5 \
5   --cv        group_kfold \
6   --n_splits  5

```

Listing 42: GroupKFold k

```

1 python src/eeg_ml_pipeline.py \
2   --mode      real \
3   --input     ./output_preproc \
4   --output    ./ml_results_kfold10 \
5   --cv        group_kfold \
6   --n_splits  10

```

Listing 43: GroupKFold k

```

1 python src/eeg_ml_pipeline.py \
2   --mode      synthetic \
3   --n_subjects 10 \
4   --output    ./ml_results_synthetic

```

Listing 44: Modalita' sintetica standalone

16.5 Riepilogo argomenti eeg_ml_pipeline.py (uso standalone)

Table 17: Argomenti accettati da eeg_ml_pipeline.py (uso diretto)

Argomento	Tipo	Descrizione	Default
-mode	str	real o synthetic	synthetic
-input	str	Dir. output preprocessing (epoche)	./output
-output	str	Dir. output risultati ML	./ml_results
-cv	str	loso o group_kfold	loso
-n_splits	int	k per GroupKFold	5
-n_jobs	int	Processi paralleli (-1 = tutti i core)	-1
-n_subjects	int	Soggetti sintetici (solo synthetic)	10
-no_save_models	flag	Non salvare i modelli .joblib	disattivo

16.6 File di output prodotti dalla pipeline ML

Table 18: File di output prodotti nella directory `ml_results/`

File	Contenuto e formato
<code>results_summary.csv</code>	Tabella con colonne <code>{metrica}_mean</code> e <code>{metrica}_std</code> per ogni modello (LR, SVM, RF)
<code>feature_matrix.npy</code>	Matrice feature $\mathbf{X}$, shape $(N_{\text{epochs}}, 105)$, dtype float32
<code>feature_names.csv</code>	Lista di 105 nomi feature: <code>Fz_theta</code> , <code>O1_alpha</code> , ...
<code>feature_ranking.csv</code>	Per tutte le 105 feature: <code>feature_name</code> , <code>rf_importance</code> , <code>rf_importance_std</code> , <code>lr_coef</code>
<code>confusion_matrices.png</code>	CM normalizzata aggregata LOSO, 1×3 subplot con conteggi assoluti
<code>roc_curves.png</code>	ROC media $\pm$ std su 300 punti, 3 modelli sovrapposti, linea chance
<code>feature_importance.png</code>	Top-20 feature: importanza Gini (RF) e coefficienti $ \beta $ (LR)
<code>metrics_boxplot.png</code>	Violin ($N \geq 8$) o box plot delle 5 metriche per fold, 3 modelli
<code>saved_models/*.joblib</code>	Pipeline sklearn complete addestrate sull'intero dataset

17 Risultati Ottenuti e Confronto con la Letteratura

17.1 Statistiche del dataset preprocessato

La pipeline è stata eseguita sul dataset reale di Chávez-Sánchez et al. (2026). Su 59 soggetti totali nel dataset Mendeley, 55 sono stati elaborati correttamente; 4 cartelle risultano mancanti nel dataset scaricato (probabile problema di download parziale).

Table 19: Statistiche aggregate del dataset elaborato dalla pipeline

Parametro	Valore
Soggetti elaborati	55 / 59
Soggetti mancanti	4 (cartelle assenti nel download)
Trial totali	$55 \times 200 = 11.000$
Epoche per soggetto	200 (100% trial validi per ogni soggetto)
Shape tensore per soggetto	(200, 21, 513)
Shape feature matrix $\mathbf{X}$	(11.000, 105) – 21 canali $\times$ 5 bande
Trial advantageous (media)	≈ 101 (50.5%)
Trial disadvantageous (media)	≈ 99 (49.5%)
Durata media registrazione	≈ 1056 s (≈ 17.6 min)
Fold LOSO	55 (uno per soggetto)
Campioni training per fold	≈ 10.800 (54×200 trial)
Campioni test per fold	200 (1 soggetto)
Componenti ICA rimosse (s-01)	2 (componenti 0 e 1, artefatti oculari)

17.2 Variabilità inter-soggetto nella distribuzione delle scelte

I dati mostrano una marcata variabilità inter-soggetto nella proporzione di scelte advantageous/disadvantageous, coerente con la letteratura IGT. Steingroever et al. (2013) riportano

che una quota significativa di partecipanti sani non sviluppa la preferenza per C/D entro 200 trial.

Table 20: Esempi di variabilità inter-soggetto nella distribuzione delle scelte

Categoria	Soggetto	Adv/Disadv	Interpretazione
Apprendimento rapido	s-51	167 / 33	Forte preferenza per C/D già sviluppata
Apprendimento rapido	s-27	159 / 41	Idem
Apprendimento rapido	s-17	160 / 40	Idem
Apprendimento lento	s-36	59 / 141	Preferenza persistente per A/B
Apprendimento lento	s-13	62 / 138	Idem
Apprendimento lento	s-03	66 / 134	Idem
Distribuzione bilanciata	(maggioranza)	$\approx 100/100$	Apprendimento parziale

La variabilità nella distribuzione ha implicazioni dirette per la classificazione: i soggetti con distribuzione estrema (s-51, s-36) producono fold LOSO con test set sbilanciato, rendendo la ROC-AUC particolarmente importante come metrica robusta (Accuracy sarebbe distorta).

17.3 Risultati LOSO – media $\pm$ std su 55 fold

Table 21: Risultati della cross-validation LOSO per i tre modelli (55 fold, 55 soggetti)

Modello	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.521 ± 0.098	0.529 ± 0.179	0.520 ± 0.277	0.475 ± 0.188	0.545 ± 0.063
SVM (RBF)	0.520 ± 0.099	0.534 ± 0.165	0.451 ± 0.289	0.431 ± 0.199	0.553 ± 0.068
Random Forest	0.513 ± 0.084	0.529 ± 0.133	0.546 ± 0.259	0.492 ± 0.159	0.532 ± 0.059

Commento ai risultati:

- **Performance sopra il caso:** tutti e tre i modelli ottengono $AUC > 0.5$ in modo consistente, confermando che la PSD pre-decisionale contiene informazione discriminante sulla scelta imminente.
- **SVM: massima AUC (0.553):** il kernel RBF cattura relazioni non lineari tra le feature PSD che il modello lineare non può rappresentare.
- **Random Forest: massimo Recall (0.546):** tende a identificare più trial advantageous correttamente, al costo di più falsi positivi (precision più bassa).
- **LR: miglior compromesso:** accuracy e AUC comparabili a SVM con interpretabilità completa tramite coefficienti β .
- **Elevata deviazione standard:** l'ampia variabilità ($\sigma_{AUC} \approx 0.06$) riflette la forte eterogeneità inter-soggetto: alcuni soggetti sono classificabili con $AUC \approx 0.65$, altri con $AUC \approx 0.50$ (non distinguibili dal caso).

17.4 Confronto con la letteratura EEG-IGT

Table 22: Confronto con gli studi EEG-IGT citati nel paper originale

Studio	Metodo principale	AUC / Acc	Note
Balconi & Angioletti (2022)	SVM, feature spettrali, cross-subj.	0.55–0.62	Setting analogo al nostro
Bianchin & Angrilli (2011)	ERP (DPN), single-trial	N/A	Feature ERP, non PSD
Cui et al. (2013)	ERP + theta Fz, averaging	N/A	Non classificazione ML
Aram et al. (2019)	Review, theta frontale	N/A	Meta-analisi
Questa pipeline (2026)	PSD Welch, LOSO, LR/SVM/RF	0.532–0.553	55 sogg., solo PSD

I risultati ottenuti (AUC 0.53–0.55) sono nel range inferiore di Balconi & Angioletti (0.55–0.62). La differenza è coerente con:

- **Maggiore severità della LOSO pura** rispetto a split random cross-subject;
- **Uso di sole feature PSD** senza feature temporali (ampiezza ERP), di connettività (coerenza) o time-frequency (wavelet);
- **Maggior numero di soggetti** (55 vs tipicamente 20–30 negli studi di confronto), che include soggetti con pattern EEG meno discriminabili.

Il paper originale [1] non include benchmark di classificazione ML (è un data descriptor focalizzato sulla validazione ERP del dataset), quindi non è possibile un confronto diretto.

18 Scelte Implementative e Motivazioni

18.1 PSD invece di analisi ERP per la classificazione

Il paper valida il dataset tramite analisi ERP (FRN, P300, N400 in Fig. 9), usando averaging su tutti i soggetti. La pipeline usa PSD per la classificazione ML single-trial. La motivazione è principalmente statistica: l'averaging ERP aumenta il SNR di un fattore $\sqrt{N}$ (dove N è il numero di trial mediati). Con $N = 100$ trial per classe per soggetto, l'SNR dell'ERP mediato è $10\times$ superiore all'SNR di singolo trial. Le feature ERP di singolo trial sono quindi troppo rumorose per una classificazione affidabile in LOSO. La PSD su finestra 2s, grazie al metodo di Welch, produce feature più stabili.

18.2 Classificazione binaria A/B vs C/D

Il dataset registra la scelta per singolo deck (A, B, C, D). La pipeline aggrega A e B come classe 0 e C e D come classe 1. La motivazione neuroscientifica è la teoria IGT di Bechara et al. (1994): la distinzione clinicamente rilevante è tra comportamento impulsivo (A/B, alta ricompensa immediata, perdita netta) e comportamento ottimale (C/D, ricompensa moderata, guadagno netto). La Fig. 7 del paper usa questa dicotomia come metrica principale di performance comportamentale. Un'alternativa sarebbe la classificazione a 4 classi (A vs B vs C vs D), ma con ≈ 50 trial per deck per soggetto il numero di campioni per classe sarebbe insufficiente per la LOSO.

18.3 Gestione dello sbilanciamento: `class_weight='balanced'`

Sebbene la distribuzione media sia quasi bilanciata (50.5% vs 49.5%), la distribuzione *per singolo fold* *LOSO* può essere fortemente sbilanciata (es. soggetto s-51: 167 advantageous vs 33 disadvantageous, rapporto 5 : 1). Il parametro `class_weight='balanced'` in tutti i classificatori corregge automaticamente:

$$w_i = \frac{N_{\text{tot}}}{2 \cdot N_{y_i}}, \quad y_i \in \{0, 1\}$$

aumentando il peso dei campioni della classe minoritaria e prevenendo il bias verso la classe maggioritaria nel fold.

18.4 Parallelizzazione con backend `loky`

Il backend `loky` (process-based) di `joblib` è scelto invece di `threading` per superare il Global Interpreter Lock (GIL) di CPython, che impedisce il vero parallelismo a thread per codice CPU-bound. Su sistemi con K core fisici, il backend `loky` riduce il tempo di esecuzione *LOSO* di un fattore $\approx K$, rendendola praticabile su hardware consumer.

18.5 Finestra pre-decisionale $[-2\text{s}, 0\text{s}]$

La finestra cattura DPN $[-0.5\text{s}, 0\text{s}]$, SCP $[-2\text{s}, 0\text{s}]$ e attività theta frontale $[-2\text{s}, -1\text{s}]$, escludendo la finestra di feedback $[0\text{s}, +1.5\text{s}]$ per evitare contaminazione con FRN e P300.

18.6 ICA: 3000 iterazioni e tolleranza 10^{-3}

Rispetto ai valori documentati originali (1500 iterazioni, 10^{-4}), il codice usa 3000 iterazioni e tolleranza 10^{-3} . Con i valori originali, l'ICA produce `ConvergenceWarning` su dati sintetici (rumore gaussiano puro, difficile per FastICA). I valori aggiornati riducono significativamente questi warning senza impatto sulla qualità del preprocessing su dati reali.

18.7 Filtro notch con metodo FIR

Il metodo FIR (invece del classico filtro notch IIR tipo “notch-band”) è usato per preservare la risposta in fase lineare dell'intera catena di filtraggio, coerentemente con la raccomandazione MNE e con la scelta FIR per il bandpass.

18.8 Assenza di baseline correction

La baseline correction (sottrazione della media in una finestra pre-stimolo fissa) è comune nell'analisi ERP ma non applicabile qui: la finestra $[-2\text{s}, 0\text{s}]$ è interamente pre-decisionale e non esiste una finestra pre-stimolo separata da usare come baseline. La normalizzazione z-score sull'intera registrazione svolge una funzione analoga, rimuovendo il livello DC e la variabilità inter-soggetto.

18.9 Libreria MNE-Python come framework EEG

MNE-Python è scelta per: essere lo standard industriale per analisi EEG/MEG in Python (> 1000 citazioni scientifiche), offrire strutture dati con gestione automatica di metadati e timing (`Raw`, `Epochs`, `Evoked`), implementare algoritmi validati dalla comunità scientifica e produrre array NumPy direttamente compatibili con `scikit-learn`.

18.10 Fallback sklearn.utils.parallel / joblib

Il modulo `eeg_ml_pipeline.py` usa un meccanismo di fallback per la parallelizzazione:

```
1 try:
2     from sklearn.utils.parallel import Parallel, delayed
3 except ImportError:
4     from joblib import Parallel, delayed
```

Questo garantisce compatibilità con scikit-learn ≥ 1.3 (che ha integrato `Parallel` nella propria API) e versioni precedenti che richiedono `joblib` diretto.

19 Conclusioni e Sviluppi Futuri

19.1 Riepilogo

La pipeline end-to-end implementata costituisce un sistema completo e riproducibile per la classificazione delle decisioni nell'Iowa Gambling Task a partire dal segnale EEG grezzo. I punti chiave dell'implementazione:

1. **Preprocessing robusto (Parte I):** bandpass FIR 0.5–70 Hz + notch + CAR + ICA FastICA ($K = 15$, 3000 iter) + normalizzazione z-score – produce epoche pulite (200, 21, 513) per soggetto, pronte per classificazione.
2. **Feature extraction PSD (Parte II):** metodo di Welch su 5 bande ($\delta, \theta, \alpha, \beta, \gamma$) per 21 canali = 105 feature per epoca, coerenti con i correlati neurali identificati dalla letteratura EEG-IGT.
3. **Tre classificatori complementari:** Logistic Regression (baseline lineare, interpretabile, β coefficienti), SVM RBF (massima AUC = 0.553, non-lineare), Random Forest (feature importance Gini).
4. **Validazione cross-subject rigorosa (LOSO):** 55 fold indipendenti, assenza garantita di data leakage inter-soggetto, stima fedele della generalizzazione a nuovi individui.
5. **Performance nel range atteso:** AUC 0.53–0.55, coerente con la letteratura per classificazione trial-by-trial EEG-IGT con sole feature PSD e LOSO pura.
6. **Parallelizzazione efficiente:** riduzione 4–6× del tempo di esecuzione su CPU multi-core tramite `joblib.Parallel` con backend `loky`.
7. **Orchestrazione unificata (run_pipeline.py):** singolo comando end-to-end in modalità sintetica o reale, con output terminale premium `rich` e logging unificato.
8. **Output scientifico completo:** CSV con metriche aggregate, PNG publication-quality, modelli `.joblib` per deployment.

19.2 Sviluppi futuri

- **Feature di connettività:** coerenza theta Fz–Cz come proxy della comunicazione prefrontale-cingolata; asimmetria alpha frontale ($F_{4,\alpha} - F_{3,\alpha}$) come indice di dominanza emisferica nella deliberazione.
- **Feature time-frequency:** wavelet di Morlet (adattive per segnali non-stazionari) e trasformata di Hilbert-Huang per catturare la dinamica temporale dell'oscillazione theta durante la deliberazione.

- **Deep learning:** CNN 1D sulle epoche raw (senza feature engineering manuale), LSTM per sfruttare la sequenza temporale delle decisioni, EEGNet come architettura specifica per EEG.
- **Ottimizzazione iperparametri:** grid search o Bayesian optimization (Optuna) per C , γ (SVM), `n_estimators`, `max_depth` (RF), C (LR) all'interno della LOSO.
- **Esclusione elettrodi auricolari A1/A2:** verificare sistematicamente l'impatto dell'esclusione di A1 e A2 dalla feature matrix, data la loro prominenza nell'analisi di feature importance RF.
- **Replicazione analisi ERP del paper:** implementazione del pipeline ERP (`mne.Evoked`, averaging per classe e per soggetto) per confronto diretto con Fig. 9 del paper (FRN, P300, N400).
- **Correzione notch filter a 60 Hz:** verifica della PSD del dataset reale preprocessato con `NOTCH_FREQ=50` e aggiornamento a 60 Hz se presente picco residuo.
- **Analisi per gruppo ENG vs PCS:** separare i risultati LOSO per gruppo sperimentale per investigare differenze cognitive tra studenti di Ingegneria e di Scienze motorie.
- **Sensor fusion:** analisi del contributo relativo di sottoinsiemi di canali (solo frontali, solo parietali, solo occipitali) per identificare la configurazione ottimale di elettrodi.

References

- [1] Chávez-Sánchez M., Torres-Ramos S., Román-Godínez I., Salido-Ruiz R.A. *Behavioral and electroencephalographic dataset simultaneously acquired during the Iowa Gambling Task*. Scientific Data, 13:359, 2026. DOI: 10.1038/s41597-026-06662-0
- [2] Bechara A., Damasio A.R., Damasio H., Anderson S.W. *Insensitivity to future consequences following damage to human prefrontal cortex*. Cognition, 50:7–15, 1994.
- [3] Bianchin M., Angrilli A. *Decision preceding negativity in the Iowa Gambling Task: an ERP study*. Brain and Cognition, 75:273–280, 2011.
- [4] Cui J.F., Chen Y.H., Wang Y., Shum D.H., Chan R.C. *Neural correlates of uncertain decision making: ERP evidence from the Iowa Gambling Task*. Frontiers in Human Neuroscience, 7:776, 2013.
- [5] Balconi M., Angioletti L. *Neurophysiology of gambling behavior and internet use vulnerability: a comparison between behavioral and EEG measures*. Clinical EEG and Neuroscience, 53:268–277, 2022.
- [6] Aram S. et al. *The Iowa gambling task: A review of the historical evolution, scientific basis, and use in functional neuroimaging*. Sage Open, 9(3), 2019.
- [7] Steingroever H. et al. *Performance of healthy participants on the Iowa Gambling Task*. Psychological Assessment, 25:180, 2013.
- [8] Gramfort A. et al. *MEG and EEG data analysis with MNE-Python*. Frontiers in Neuroscience, 7:267, 2013.
- [9] Pedregosa F. et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12:2825–2830, 2011.
- [10] Nunez P.L. et al. *A theoretical and experimental study of high resolution EEG based on surface Laplacians and cortical imaging*. Electroencephalography and Clinical Neurophysiology, 90:40–57, 1994.
- [11] Kóbor A. et al. *Different strategies underlying uncertain decision making: Higher executive performance is associated with enhanced feedback-related negativity*. Psychophysiology, 52:367–377, 2015.